

Bachelorarbeit

OSS Risk Radar: Risk Assessment for Open Source Dependencies

zur Erlangung des akademischen Grades
'Bachelor of Science' B.Sc.
im Studiengang 'Informatik'

vorgelegt dem Fachbereich Informatik und Wirtschaftsinformatik der
Provadis School of International Management and Technology
von

Eric Krause
Matrikelnummer: D720
BIN-T23-F-4
eric.krause@telekom.de

Erstgutachter: Prof. Dr. Lamya Abdullah

Zweitgutachter: Dr. Peter Manshausen

Ende der Bearbeitungsfrist: 29.07.2026

Abstract

Modern software depends heavily on open-source components, yet downstream projects remain exposed when an upstream repository quietly stops being maintained. This thesis asks to what extent publicly observable repository health and maintenance indicators can predict whether an open-source dependency becomes inactive within a twelve-month horizon, and how the resulting risk score can support risk-based dependency triage. Following a Design Science Research approach, the work develops and evaluates OSS Risk Radar, an artifact that constructs a leakage-controlled historical dataset from GH Archive event data and public package metadata, engineers observation-time activity, responsiveness, and contributor-concentration features, and trains calibrated inactivity-risk models.

On a repository foundation dataset of 5,000 projects and 50,000 quarterly observation snapshots, a gradient-boosted model reaches a held-out ROC-AUC of 0.958 with a Brier score of 0.078, and its skill largely persists under a repository-disjoint split (0.950). This aggregate figure is, however, a poor measure of what the model contributes: a trivial rule that ranks repositories by days since their last commit reaches 0.957 on the same rows, against 0.974 for the model. The learned model earns its complexity on the repositories that are still active at the observation time — the only ones for which an early warning has any decision value — where it attains 0.905 against 0.844 for the best trivial rule. The thesis therefore reports both a null result on the aggregate comparison and a positive result on early decline. A human-actor filter is validated by ablation, and security-practice signals are kept as parallel triage context rather than as predictors of inactivity. A practical demonstrator applies the deployed model per repository to the dependency repositories of a real project, producing a calibrated, confidence-aware, and evidence-backed ranked triage view. The score is positioned as a decision-support signal for prioritizing dependency review, not as a verdict on abandonment.

Contents

Abstract	II
List of Figures	VI
List of Tables	VII
List of Abbreviations	IX
1 Introduction	1
1.1 Motivation and Problem Statement	1
1.2 Research Question and Objectives	2
1.3 Scope and Assumptions	3
1.4 Contributions	3
1.5 Structure of the Thesis	4
2 Background	5
2.1 Software Supply-Chain Security	5
2.2 Open-Source Sustainability and Project Health	6
2.3 Metric Pitfalls and Limitations	7
2.4 Machine Learning Basics for Tabular Risk Scoring	7
3 Related Work	9
3.1 OSS Inactivity and Abandonment Operationalizations	9
3.2 Repository Health Metrics and Dashboards	10
3.3 Security-Practice Indicators	10
3.4 Responsiveness Measurement Caveats	11
3.5 Research Gap	11
4 Methodology	12
4.1 Research Design	12
4.2 Iterative Artifact Development	13
4.3 Definitions and Prediction Target	13

4.4	Data Sources	15
4.4.1	Offline Repository Metadata and Its Consequences	15
4.5	Dataset Construction Workflow	16
4.6	Observation Window and Historical Coverage	16
4.7	Feature Engineering	17
4.7.1	Undefined Versus Zero-Valued Features	18
4.8	Full-History and Cold-Start Prediction Regimes	18
4.9	Leakage Prevention	19
4.10	Model Training	20
4.11	Evaluation Strategy	21
4.12	Methodological Limitations	22
5	Implementation	23
5.1	System Architecture Overview	23
5.2	Historical Dataset Builder	24
5.2.1	Label-Completeness Correction	25
5.3	Model Training and Evaluation Instrumentation	26
5.4	Reproducibility of the Evaluated Artifact	27
6	Results and Discussion	28
6.1	Predictive Performance	28
6.1.1	Comparison Against Non-Learned Baselines	30
6.1.2	How Much Is Reconstructed History Worth?	30
6.2	Calibration and Threshold Selection	31
6.3	Feature Importance and Error Analysis	32
6.3.1	Feature Attribution	32
6.3.2	Robustness to Repository Recurrence	33
6.3.3	Slice Evaluation	33
6.3.4	Error Analysis	34
6.4	Ablation: The Human-Actor Filter	35
7	Practical Demonstrator	36
7.1	Example Repository Set	36
7.2	Risk Ranking Output and Interpretation Guidance	37
8	Threats to Validity	40
8.1	Construct Validity	40
8.2	Internal Validity	40
8.3	External Validity	41

8.4 Conclusion Validity	42
9 Conclusion and Outlook	43
9.1 Outlook and Future Work	44
Bibliography	46
A Appendix	49
A.1 Observable Health Indicator Categories	49
A.2 Full-History Feature Reference	50
A.3 Foundation Seed Composition	57
A.4 Undefined Feature Handling	57
A.5 Implementation Detail	58
A.5.1 Technology Selection	59
A.5.2 Dataset-Builder Modules	60
A.5.3 Dataset-Quality Reporting	61
A.5.4 Model Configurations	61
A.5.5 Artifact Serialization	62
A.5.6 Training Orchestration	62
A.5.7 Artifact Staging and Promotion Gate	63
A.5.8 Scoring Service	64
A.5.9 Backend Orchestration Service	64
A.5.10 Analyst Frontend	65
A.5.11 Deployment	66
A.5.12 Feature-Set Refinement	66
A.5.13 Slice Evaluation	67
A.5.14 Repository-Disjoint Robustness Split	68
A.6 Dataset Construction Steps	69
A.7 Evaluation Metric Definitions	69
A.8 Reproducibility	71
A.9 Full Demonstrator Ranking	73
A.10 Supplementary Evaluation Tables	73
A.10.1 Human-Actor Filter Ablation	73
Declaration on the Use of Artificial Intelligence	76
Declaration of Authorship	78

List of Figures

List of Tables

6.1	Held-out test metrics by model family and feature regime, with the two trivial baselines. Higher is better for ROC-AUC, F1, and quality; lower is better for Brier and ECE. The best value within each regime is shown in bold. The recency baseline is defined on the 4,556 test rows that have an observable commit history; see the text for the like-for-like comparison.	29
6.2	Held-out ROC-AUC by feature-set size, on identical row sets.	31
7.1	Flagged dependency repositories of <code>ArchipelagoMW/Archipelago</code> (the two <code>replace_candidate</code> items and the four <code>review</code> items), with the single full-history repository shown for contrast. Risk is the 0–100 inactivity-risk score; “Conf.” is the per-prediction confidence; “Regime” is full-history (FH) or cold-start (CS). The complete 30-repository ranking is in Appendix A.9.	38
A.1	Categories of observable health indicators and their expected relevance	49
A.2	Implemented full-history feature groups	51
A.3	Reconstructed historical feature reference (<code>feature-set-v4-full-history</code>): exact definition and rationale for each of the 43 historical model inputs, plus the two diagnostic-only variables <code>repo_archived_at_obs</code> and <code>repo_age_days</code>	51
A.4	Current-snapshot feature reference: the 17 features obtainable without event history. These form the complete cold-start vector and are also included in the full-history vector.	56
A.5	Foundation seed strata: target frame versus realized seed (5,000 repositories).	57
A.6	Implemented system components	59
A.7	Dataset builder modules	60
A.8	Historical dataset construction workflow	69

A.9	Complete ranked inactivity-risk view for the dependency repositories of ArchipelagoMW/Archipelago, as produced by the deployed model. Risk is the 0–100 inactivity-risk score; “Conf.” is the per-prediction confidence; “Regime” is full-history (FH) or cold-start (CS).	74
A.10	Held-out ROC-AUC and Brier score of xgboost-full-history within subgroups of the test slice. “Inactive rate” is the positive rate in the subgroup.	74
A.11	Cross-test of the human-actor filter: training on bot-contaminated features versus clean features, both evaluated against the identical clean (human-filtered) label on the held-out slice. A lower ROC-AUC or higher Brier for the contaminated features means the filter helps.	75
A.12	Commits to the OSS Risk Radar repository co-authored by Claude Opus 4.8, in chronological order. Hashes are abbreviated to ten characters. .	77

List of Abbreviations

API	Application Programming Interface
CVSS	Common Vulnerability Scoring System
ML	Machine Learning
OSS	Open Source Software

1 Introduction

1.1 Motivation and Problem Statement

Modern software systems are not developed in isolation, but are commonly assembled from first-party code, third-party components, and open-source software dependencies. In this thesis, open-source software is understood as software distributed under a license that complies with the Open Source Definition, which includes rights such as use, modification, and redistribution [1]. From a secure software development perspective, such dependencies are relevant because the NIST Secure Software Development Framework explicitly includes the acquisition and maintenance of well-secured third-party and open-source components as part of secure development practice [2]. Similarly, the SLSA framework addresses software supply-chain integrity by focusing on artifacts, build processes, provenance, and related security controls [3].

The practical relevance of dependency management becomes visible in both empirical studies and real-world incidents. Miller et al. studied open-source dependency abandonment in the npm ecosystem and found that abandonment is common among widely used packages while downstream projects often remain exposed, particularly through indirect dependencies that developers have limited ability to react to [4]; in an earlier interview study they found that affected developers often lacked clear strategies and felt they were improvising [5]. Delayed maintenance also prolongs security exposure: Prana et al. analyzed dependencies in 450 Java, Python, and Ruby projects and found that many vulnerabilities persisted throughout the observation period, taking several months to resolve on average [6].

Real-world incidents illustrate the potential impact. The 2016 unpublishing of the small npm package `left-pad` disrupted many directly and transitively dependent projects [7]; the 2017 Equifax breach exploited an unpatched Apache Struts vulnerability and led to a settlement of up to \$700 million [8], [9]; and the XZ Utils backdoor showed that maintainer trust itself can become part of the attack surface through social-engineering takeover [10].

These examples do not imply that every inactive repository is insecure, abandoned, or harmful. A project may be stable, temporarily paused, or maintained through channels that are not fully visible in public repository data. However, they show that dependency health, maintainability, and responsiveness are relevant for secure software supply chains. Therefore, this thesis treats repository inactivity not as a definitive indicator of project death, but as an operational proxy for elevated maintenance risk. The central question is whether publicly observable repository signals can help estimate this risk early enough to support dependency triage and monitoring.

1.2 Research Question and Objectives

The central research question of this thesis is:

To what extent can a machine-learning-based scoring tool, using only publicly observable repository health and maintenance indicators, predict 12-month OSS dependency inactivity, and how can this inactivity-risk score be combined with parallel security-practice signals to support risk-based dependency triage in secure dependency management?

To answer this research question, the thesis pursues the following objectives:

1. Define an operational inactivity label for open-source repositories within a 12-month prediction horizon.
2. Construct a reproducible dataset from publicly observable repository metadata and dependency-related sources.
3. Engineer observation-time activity and maintenance features for the inactivity model, and surface selected security-practice signals as parallel triage context rather than as inputs to the inactivity prediction.
4. Train and compare simple baselines and machine-learning models for inactivity-risk prediction.
5. Evaluate the models using discrimination and calibration metrics that are suitable for risk scoring.
6. Demonstrate how the trained model can be applied per repository to a set of dependency repositories to support risk-based triage.

The research follows a Design Science Research-oriented approach because the thesis develops and evaluates an artifact: an inactivity-risk scoring workflow for open-source

dependencies [11], [12]. The empirical part evaluates whether the artifact produces useful predictive signals, while the practical demonstrator shows how the scoring workflow can be integrated into dependency monitoring.

1.3 Scope and Assumptions

The scope of this thesis is limited to open-source software dependencies whose source repositories and package metadata are publicly observable. The empirical dataset focuses on repositories that can be associated with an open-source license. Where GitHub license metadata is used, matched license keys and names are treated according to GitHub’s SPDX-based license information [13]. The thesis does not investigate private repositories, closed-source vendor components, or dependencies whose source repository cannot be mapped with sufficient confidence.

The prediction target is repository inactivity within the interval $[t, t + 12 \text{ months}]$. Inactivity is operationalized primarily through the absence of commits and releases or tags during the prediction horizon. This definition follows the idea that revision activity can be used as an observable signal for survival-style analysis of open-source projects [14]. Issue and pull-request activity are treated as predictor variables instead of label components, because they can indicate maintenance interaction without necessarily changing the released software artifact. Responsiveness-related features are interpreted cautiously, since bot-generated responses can distort measurements such as time to first response in pull-request workflows [15].

The thesis assumes that publicly observable repository signals can provide useful but incomplete evidence about future inactivity. It does not assume that these signals fully explain project sustainability. This distinction is important because previous research shows that cross-project health indicators can have different meanings depending on project context [16]. As a result, the model output is interpreted as a triage aid rather than as an automated decision system.

1.4 Contributions

Realizing the objectives above, this thesis contributes an operational 12-month inactivity-risk label and a reproducible construction workflow for it; an empirical evaluation of whether public repository health and maintenance indicators support machine-learning-based inactivity prediction, benchmarked against simple baselines on both discrimination

and probability calibration, with security-practice signals kept as complementary triage context rather than as inactivity predictors; and a per-repository demonstrator that turns the score into a ranked triage view. These contributions are intended to support software teams prioritizing dependency review effort. The goal is not to replace expert judgement, but to provide a measurable and reproducible signal that helps identify dependencies deserving closer attention.

1.5 Structure of the Thesis

The remainder of this thesis is structured as follows. Chapter 2 introduces the theoretical background, including software supply-chain security, open-source sustainability, project health metrics, and machine-learning basics for tabular risk scoring. Chapter 3 discusses related work on inactivity and abandonment operationalizations, repository health metrics, and security-practice indicators, and positions the artifact relative to them. Chapter 4 presents the research design, inactivity label, dataset construction, feature engineering approach, models, baselines, and evaluation strategy. Chapter 5 describes the technical implementation of the data pipeline and scoring workflow. Chapter 6 reports and discusses the empirical results, including predictive performance, calibration, feature importance, and error analysis. Chapter 7 demonstrates the trained model applied per repository to a set of dependency repositories as a ranked triage view. Chapter 8 consolidates the threats to validity and their mitigations. Chapter 9 concludes the thesis and outlines limitations and opportunities for future work.

2 Background

This chapter introduces the conceptual foundations required for the thesis. It first explains the software supply-chain security context in which open-source dependencies are used and assessed. It then discusses open-source software sustainability and repository health metrics. Afterwards, it outlines important limitations of metric-based project assessment. Finally, it introduces the machine-learning concepts that are relevant for tabular inactivity-risk scoring.

2.1 Software Supply-Chain Security

Software supply-chain security focuses on reducing risks that arise when software artifacts, build processes, dependencies, and distribution channels are not sufficiently controlled or verifiable. Two frameworks establish why dependencies fall within this scope: the NIST Secure Software Development Framework treats the acquisition and maintenance of well-secured software components as part of secure development practice [2], and the SLSA framework defines security-oriented requirements for artifacts and build processes, including provenance and integrity controls [3]. Together they motivate monitoring dependencies beyond simple version updates.

Open-source dependencies are especially relevant because they are reused across many downstream projects. Open-source software is defined by licensing conditions that allow use, modification, and redistribution under criteria specified by the Open Source Initiative [1]; these freedoms enable broad reuse but also mean that downstream users depend on external maintainers and public infrastructure, so the maintenance activity of an upstream repository becomes relevant for downstream risk assessment. Supply-chain security thus provides the practical context for inactivity-risk scoring: the goal is not to determine whether a dependency is secure in general, but to estimate whether publicly observable repository signals indicate an elevated risk of future inactivity, supporting monitoring, replacement planning, or prioritization of manual review.

This context also explains why the unit of analysis is a single repository rather than a

dependency graph: enumerating which repositories a project depends on is the task of established software-composition-analysis tooling, so the dependency inventory is treated as an external input (Section 5.1). Supply-chain security is therefore the reason to look beyond version numbers and consider the maintenance trajectory of the upstream repositories that a project depends on.

2.2 Open-Source Sustainability and Project Health

Open-source sustainability describes whether a project can continue to provide value and maintain its software over time. Project health is commonly assessed through observable indicators such as activity, responsiveness, contributor participation, and release behavior, for which the CHAOSS project provides standardized metrics models [17].

Repository activity is one category of observable project health. Examples include commits, releases, tags, issues, pull requests, and contributor activity. These signals are useful because they can often be collected from public repository platforms and package metadata sources. Prior research has used revision activity and project-level attributes to study open-source project survival and inactivity-related outcomes [14]. In this thesis, such activity signals form the basis for predicting whether a dependency repository becomes inactive within a 12-month horizon.

Security-practice indicators form a second category of observable signals. OpenSSF Scorecard assesses open-source projects using automated security-related checks [18]. These describe repository configuration and security posture rather than future maintenance activity, and Section 3.3 sets out why this thesis keeps them as parallel triage context instead of feeding them into the inactivity model.

Project health is nonetheless broader than any single metric [19], which is why inactivity-risk scoring is treated here as a limited operational signal for dependency triage rather than as a judgement of project sustainability.

These categories of observable health indicators — commit activity, contributor activity, issue and pull-request activity, release activity, age and popularity, and security practice — motivate the feature groups introduced later; their example signals, public sources, and expected relevance to inactivity risk are summarized in Appendix A.1 (Table A.1), and the exact per-feature definitions of the implemented set are given in Section 4.7 and Appendix A.2.

2.3 Metric Pitfalls and Limitations

Although repository metrics are useful, they can be misleading when interpreted without context [16]: a metric value can mean different things depending on the project’s type, maturity, governance model, and maintenance style.

One example is commit activity. A low number of recent commits may indicate reduced maintenance, but it may also indicate that a project is stable and does not require frequent changes. Similarly, a high number of issues may indicate an active user community, but it may also indicate unresolved maintenance problems. Therefore, this thesis does not treat single metrics as direct proof of project health or project failure. Instead, multiple signals are combined and evaluated empirically.

Responsiveness metrics require particular caution, because time-to-first-response can be affected by automated bot replies in pull-request workflows [15]. Such indicators should therefore be interpreted carefully and, where possible, separated into human and automated activity (Section 3.4).

A further limitation is that public repository data may be incomplete. Development may move to another platform, releases may be published outside the observed repository, or maintainers may communicate through channels that are not captured by GitHub metadata. Package-to-repository mappings may also be ambiguous or missing. Therefore, the empirical results of this thesis must be interpreted as evidence based on publicly observable data, not as a complete representation of all project activity.

These limitations influence the design of the inactivity label. Inactivity is not defined as project death or abandonment. Instead, it is operationalized as the absence of selected observable activity signals within a defined prediction horizon. This makes the label measurable and reproducible, while still acknowledging that it is only a proxy for elevated maintenance risk.

2.4 Machine Learning Basics for Tabular Risk Scoring

The prediction task in this thesis is binary classification: at observation time t a repository is represented by features derived from public metadata and activity signals, and the model estimates the probability that it becomes inactive in $[t, t + 12 \text{ months}]$. Because inactivity is by definition closely related to recent activity, a recency-based baseline (time since the last observed commit or release) is required as a reference point; a machine-learning model is only worth reporting if it improves over it. Among

learned models, logistic regression serves as an interpretable probabilistic baseline, while tree-based models such as gradient boosting capture non-linear feature interactions [20], [21], [22], which suits tabular repository metadata where activity and maintenance indicators may interact non-linearly.

Model evaluation should consider both discrimination — how well a model separates inactive from active repositories, measured by the area under the ROC or precision-recall curve, the latter being relevant under class imbalance [23] — and calibration, whether predicted probabilities match observed outcome frequencies, which matters because a risk score should be interpretable as a probability-like signal rather than only a ranking [24]. Finally, the dataset should be split in a time-aware rather than random way, so that information from later periods cannot influence model development for earlier ones; this matches the intended use of historical information to predict future inactivity.

3 Related Work

This chapter positions the thesis relative to existing work. Chapter 2 introduced the general concepts of supply-chain security, project health metrics, and tabular machine learning. This chapter goes one step further and discusses concrete prior approaches, their operationalizations, and the gap that this thesis addresses. The aim is comparative rather than introductory: for each strand of work, the relevant difference to the present inactivity-risk artifact is made explicit.

3.1 OSS Inactivity and Abandonment Operationalizations

A first strand of work studies how open-source inactivity or abandonment can be defined and measured. Robinson et al. apply survival-style analysis to open-source Python projects and use revision activity as an observable signal for project survival [14]. This supports the idea that activity-based signals can be used to reason about future project state, but survival analysis typically estimates time-to-event over a population rather than producing a per-repository forward-looking risk score at a fixed horizon.

Miller et al. study dependency abandonment in the npm ecosystem and show that abandonment is common among widely used packages and that downstream projects often remain exposed, including through transitive dependencies [4]. Their earlier interview study shows that developers frequently lack clear strategies for responding to abandonment [5]. These works motivate the practical need for an early-warning triage signal, but they characterize abandonment retrospectively and at the ecosystem level rather than predicting a 12-month inactivity outcome for an individual repository from point-in-time features.

The present thesis differs in three ways. First, it fixes a 12-month prediction horizon and a snapshot-based observation date, so that prediction is strictly forward-looking. Second, it derives the label from a multi-signal maintenance rule rather than from a single revision-activity threshold. Third, it treats issue and pull-request activity as

predictors rather than as label components, to avoid conflating maintenance interaction with the released-artifact signal.

3.2 Repository Health Metrics and Dashboards

A second strand concerns standardized project health measurement. The CHAOSS project provides metrics models, including the Starter Project Health Metrics Model, as structured entry points for measuring project health [17]. Goggins et al. argue that making project health transparent requires structured measurement and interpretation rather than isolated indicators [19]. These efforts provide a vocabulary of observable signals that informs the feature engineering of this thesis.

However, dashboards and metrics models are primarily descriptive: they surface indicators for human interpretation rather than predicting a future outcome. Yehudi et al. show that individual context-free health indicators are insufficient for identifying sustainability across heterogeneous projects [16]. This thesis takes that critique as a design constraint: instead of presenting single indicators, it combines many observation-time signals into a calibrated predictive model and evaluates the combination empirically, while explicitly acknowledging that the resulting score is a limited operational proxy.

3.3 Security-Practice Indicators

A third strand concerns security-practice signals. OpenSSF Scorecard provides automated security-related checks for open-source repositories [18], and supply-chain frameworks such as the NIST SSDF and SLSA motivate why dependency security posture matters [2], [3]. These checks describe present-day security practices rather than future maintenance activity.

This distinction is important for the present work. As stated in Section 4.1, OpenSSF Scorecard is used as a parallel security-posture signal in OSS Risk Radar and is deliberately not part of the inactivity model feature vector. The thesis therefore does not claim that security-practice checks improve inactivity prediction; it keeps the predictive target focused on activity and maintenance signals, while presenting security posture as complementary context for triage.

3.4 Responsiveness Measurement Caveats

Several features in this thesis describe responsiveness, such as issue first-response time and pull-request response and merge latency. Hasan et al. show that bot-generated first responses can distort time-to-first-response measurements in GitHub pull requests [15]. This thesis treats responsiveness features as reconstructed proxies derived from GH Archive event state, interprets them cautiously, and does not treat them as canonical CHAOSS metrics, in line with the caveats discussed in Section 2.3.

3.5 Research Gap

Prior work provides operationalizations of inactivity and abandonment, standardized health metrics, and security-practice indicators, but it largely characterizes project state descriptively or retrospectively. The gap addressed by this thesis is a reproducible, forward-looking, calibrated per-repository inactivity-risk model with a fixed 12-month horizon, a multi-signal maintenance label, leakage-controlled point-in-time features, and an explicit separation between the inactivity prediction target and parallel security-posture signals.

4 Methodology

This chapter describes the methodological design used to answer the research question. The thesis follows a Design Science Research-oriented approach, because the work develops and evaluates an artifact for a practical software engineering problem: an inactivity-risk scoring workflow for open-source dependencies [11], [12]. The artifact is implemented as OSS Risk Radar, a decision-support tool for open-source dependency maintenance and software supply-chain risk triage [25].

The methodology combines artifact construction with historical supervised learning. Instead of using manually assigned or synthetic labels, the training dataset is built from real open-source repositories and historical GitHub event data. For each repository, the dataset builder creates observation snapshots at defined points in time. Each snapshot contains only features that would have been available at or before the observation date t . The label is derived from the following 12-month period $(t, t + 12 \text{ months}]$. This design reflects the intended use case: estimating future inactivity risk from observable signals, rather than explaining inactivity after it has already occurred.

4.1 Research Design

The research design consists of three connected parts. First, OSS Risk Radar is developed as a software artifact that can ingest dependency or repository information, enrich it with public signals, and produce inactivity-oriented risk scores. Second, a historical supervised training dataset is constructed from real open-source repositories. Third, trained model artifacts are evaluated quantitatively and qualitatively to determine whether the scores are useful for risk-based dependency triage.

The artifact is not a vulnerability scanner or a definitive trust score, and does not claim that a repository is secure or insecure; it estimates whether observable maintenance and activity signals indicate elevated inactivity risk. Because inactivity is only one aspect of dependency risk — a repository can be inactive without being vulnerable, or active while still having security weaknesses — the model output is interpreted as a

decision-support signal for prioritization and manual review.

The scope of the predictive model must be stated precisely, because the research question also refers to security-practice signals. OpenSSF Scorecard is used as a *separate* security-posture signal in the surrounding enrichment workflow [18], *not* as part of the predictive feature vector: the inactivity classifier is trained exclusively on activity, maintenance, responsiveness, and concentration signals, and security-practice indicators are presented to the analyst as a parallel posture output rather than as predictors of inactivity. This separation avoids overclaiming that security-practice checks improve inactivity prediction and keeps the predictive target conceptually clean.

4.2 Iterative Artifact Development

OSS Risk Radar was developed iteratively. The first iteration validated the end-to-end workflow with a small pilot training run — creating historical snapshots, training a model, exporting an artifact, and returning scores for manual inspection — and is not used in any reported result. Manual inspection exposed implausible scoring in some cases: one operationally active project scored implausibly high and others scored low where a more nuanced assessment seemed warranted. Treated as formative error analysis rather than as a measure of accuracy, these cases drove a revision of the feature set and training setup to better separate activity, release behavior, contributor dynamics, responsiveness, backlog pressure, and concentration risk — the build–evaluate–diagnose–refine cycle at the core of the Design Science Research framing.

The second iteration produced the dataset and model artifacts used for the reported evaluation, expanding the empirical basis to the repository-first foundation dataset of Section 4.4, whose seed strata serve only as sampling provenance while the label is derived from future repository behavior.

4.3 Definitions and Prediction Target

The central unit of analysis is a GitHub repository. In the training dataset, a row represents a repository observation snapshot at a specific observation date t . The prediction task is binary classification:

Given the observable repository and package state at time t , predict whether the repository becomes inactive within the following 12 months.

The exported training label is named `label_inactive_12m`. Internally, the dataset builder first computes a maintenance variable named `maintained_12m`. The inactivity label is then derived as the inverse of this maintenance variable. If the future window is incomplete, the row remains unlabeled and is not used for supervised model fitting.

The future label window is defined as:

$$(t, t + 12 \text{ months}]$$

The current implementation treats a repository as maintained within the 12-month horizon if it is not archived or deleted by $t + 12$ months and if at least two of the following future-window activity checks are fulfilled:

1. human commits occur in at least three distinct months,
2. at least two contributors are active,
3. at least one release or package version publication occurs,
4. at least two pull requests are merged.

This definition deliberately avoids treating a single event as sufficient evidence of maintenance. At the same time, it remains operationally measurable from historical public data. The merged-pull-request condition is used as a practical proxy for broader maintainer interaction, because GH Archive data does not reliably identify all maintainer responses without additional issue-comment attribution logic [26].

One qualification applies to the archival condition in the reported dataset. Because the dataset is built through the reproducible offline path, which supplies no `archived_at` timestamp (Section 4.4.1), the “not archived or deleted” condition never fires and the label reduces to the two-of-four activity rule alone. This is a conservative simplification rather than a distortion — an archived repository receives no commits, contributors, releases, or merged pull requests in its future window, so the activity rule already marks it inactive — but it does mean a repository archived while still receiving activity would not be caught by the archival condition.

The specific thresholds in this rule (the two-of-four requirement and the numeric cut-offs for months, contributors, releases, and merged pull requests) are heuristic design choices rather than externally validated constants. They were chosen to require sustained rather than incidental activity. Because the label definition directly influences class prevalence, the sensitivity of the results to these thresholds is treated as a robustness concern and discussed in Section 4.12.

4.4 Data Sources

The dataset construction uses public and reproducible data sources wherever possible. GH Archive is used as the primary source for historical GitHub event data. GH Archive provides public GitHub event streams as hourly archives [27]. The dataset builder parses these archives into normalized repository histories, including commits, issues, pull requests, releases, stars, and forks.

Repository and package metadata are enriched through multiple adapters. `deps.dev` is used for package and dependency metadata where package-to-repository mapping is required [28]. GitHub metadata is used for stable repository fields such as creation time, default branch, and fork state. OpenSSF Scorecard is used by the broader OSS Risk Radar enrichment workflow as a separate security-practice signal source [18], but, as stated in Section 4.1, it is not part of the historical inactivity feature vector. Present-day GitHub activity totals are not used as historical observation-time features, because they would leak information from after the observation date into older snapshots.

The repository-first foundation dataset is drawn from a generated seed of public, non-fork GitHub repositories that have a license object and meet a minimum star threshold (default 100 stars), stratified into active, dormant, and archived buckets that serve only as sampling provenance: the supervised labels come exclusively from future activity in the 12-month window [26]. The sampling frame is designed to oversample inactive repositories, targeting 45% active against 55% dormant or archived. The realized seed departs from this target, because the dormant and low-star active strata could not be filled from the search frame and a fallback stratum topped the seed up with active repositories: as built it is 52.0% active, 28.0% dormant, and 20.0% archived (Appendix A.3), so the intended oversampling is only partly realized at the seed level. The inactive base rate that matters is instead the one observed in the held-out test slice, reported in Chapter 6; because it lies well above the natural OSS inactivity rate, predicted probabilities are calibrated to this sampled prevalence rather than to a specific deployment population, a point revisited for calibration and precision in Section 4.11 and Section 4.12.

4.4.1 Offline Repository Metadata and Its Consequences

The reported dataset is built through the offline path: repository identities come from the seed and no live GitHub, registry, or `deps.dev` call is made during construction, so the build depends only on the archived seed and the GH Archive files and can be repeated exactly without credentials, rate limits, or drift in mutable metadata. Because every

observation-time feature is reconstructed from the event history rather than fetched, the predictive content is unaffected. The path does leave the repository creation date, archival timestamp, and fork flag unpopulated; the three resulting consequences — an identically zero `repo_age_days`, an inert archival label condition (Section 4.3), and an inert fork filter — are detailed in Appendix A.4, and populating these attributes is noted as future work in Chapter 9.

4.5 Dataset Construction Workflow

The historical dataset builder creates an inspectable dataset through a sequence of intermediate steps: it loads repository candidates from the foundation seed, resolves repository identities, enriches stable metadata through GitHub, parses GH Archive event files into normalized repository histories, builds quarterly observation snapshots, computes observation-time features using only data available at or before t , computes 12-month labels using only the future window $(t, t+12 \text{ months}]$, and exports the training snapshot dataset together with a repository feature cache for runtime full-history scoring. The individual steps and the intermediate files written at each stage — which make the construction process auditable and allow the dataset to be inspected before training — are listed in Appendix A.6. The final export follows the existing training snapshot contract used by OSS Risk Radar, so the historical dataset builder can feed the notebook-based training pipeline without requiring a separate model-training interface. The commands that regenerate the seed and rebuild the dataset, the per-run documentation items, and the dataset hash of the reported run are given in Appendix A.8.

4.6 Observation Window and Historical Coverage

The full-history training run used in the reported evaluation contains 50,000 rows: 5,000 repositories observed at ten quarterly dates from 2023-01-01 to 2025-04-01. The GH Archive files available for the run span 2021-01 to 2026-06-16, so the twelve-month label window of even the latest observation date (ending 2026-04-01) lies fully inside the coverage horizon, and all 50,000 rows are labeled. The rows are split into 37,500 training rows, 7,500 validation rows, and 5,000 held-out test rows. Because the cohorts are equally sized, the 10% test partition coincides exactly with the final observation date: every held-out row is observed at 2025-04-01. This yields a clean temporal holdout but means that every test repository also appears in the training slice at an earlier date, which is why the repository-disjoint robustness split of Section A.5.14 is

reported alongside it.

A snapshot serves as a labeled example only if the historical data covers its full 12-month future window; otherwise the row stays unlabeled, because the absence of future activity is meaningful only when that period is actually observed, and sparse or partial GH Archive coverage would otherwise be misread as inactivity. (A smaller subset is used only for local pipeline smoke testing, not for reported results.)

4.7 Feature Engineering

The feature engineering step converts repository and package history before t into numeric variables. The reported full-history model uses feature set **feature-set-v4-full-history**, whose 60 inputs combine 43 reconstructed historical features — spanning commit and contributor activity, issue and pull-request flow, release cadence, derived risk proxies, and responsiveness and backlog pressure — with 17 current-snapshot features that require no event history, such as dependency context, popularity, and ecosystem. The cold-start regime (Section 4.8) uses exactly the latter 17. The feature groups, and the exact definition and rationale for every individual feature, are documented in the full feature reference in Appendix A.2 (Tables A.2, A.3, and A.4).

The features are derived from a normalized event history rather than any single source field: GH Archive supplies typed events that the builder parses into a per-repository chronological history of commits, issues, pull requests, releases, stars, and forks [27]. Each observation-time feature is a trailing-window aggregate anchored at t — counts over the 30, 90, and 365 days before t , state values at t , and derived ratios, latencies, and drop-off measures. Trailing windows are preferred to cumulative totals because the target is trajectory rather than lifetime volume, and two conventions keep variables comparable across heterogeneous projects: commit- and contributor-based counts apply a human-actor filter so that automated activity does not inflate the maintenance evidence, and absolute counts that would not compare across project sizes are also provided in a relative or trend-based form.

The selection of these signals is grounded in prior work: revision activity predicts project survival [14], motivating the commit, contributor, and release windows, while the issue, pull-request, and responsiveness signals follow the observable-metric vocabulary of the CHAOSS project health model [17], [19]. Because context-free indicators are individually insufficient across heterogeneous projects [16], the design learns a calibrated combination of many observation-time signals rather than relying on any single metric, and responsiveness features are treated as reconstructed proxies rather than canonical

CHAOSS metrics, since bot-generated responses can distort them [15].

Some variables are lower-bound proxies rather than complete measurements: stars and forks reconstructed from GH Archive undercount history if the archive window starts after the repository was created, and reconstructed issue-backlog and pull-request response features should not be read as canonical CHAOSS metrics.

4.7.1 Undefined Versus Zero-Valued Features

A feature that cannot be computed for a repository must be distinguished from one whose value is genuinely zero, because for several variables zero is the *most favourable* value rather than a neutral one. A quantity is *right-censored* when a repository has no observed commit — the “days since last commit” has no finite value but is at least the age of the observable record, so the censoring bound encodes it as maximally stale rather than as freshly committed. A quantity is *undefined* when it does not exist at all — the median time to merge a pull request is meaningless for a repository that merged none — and is left for the model artifact to impute rather than set to a misleading zero. The exact censoring bound, the per-model imputation rules, and the two indicator features that carry the “no events at all” signal are given in Appendix A.4.

This distinction is not cosmetic: encoding the absence of activity as a zero-valued latency makes the absence itself predictive through a variable whose documented meaning is responsiveness, so a model can appear to have learned that fast responders fail when it has in fact learned that silent projects fail. Keeping the two apart is what allows the feature attributions of Section 6.3.1 to be read as evidence about maintenance behaviour rather than data availability. The feature-set revision after the first manual review (Section 4.2) also excludes features that would not be reliable point-in-time predictors: `repo_archived_at_obs`, for example, is retained as diagnostic metadata while already-archived rows are filtered out during fitting, so that archival cannot become a shortcut label.

4.8 Full-History and Cold-Start Prediction Regimes

Every model is trained twice, on two nested feature sets. The *full-history* regime uses all 60 inputs, including the 43 features reconstructed from the GH Archive event history before t — time-windowed commit, release, contributor, issue, and pull-request signals, and the activity-drop proxies derived from them. The *cold-start* regime uses only the 17 current-snapshot features, which require a single observation of the repository and

its package metadata and no event history at all.

The pairing is an ablation on *feature availability*, and because GH Archive is public it is worth stating precisely what it measures. The historical features are obtainable in principle for any public repository, so the regimes do not contrast what a scorer *can* know with what it *cannot*. They measure what the reconstruction is *worth*: whether parsing tens of thousands of archive files buys predictive skill that a single metadata read does not already supply. The comparison is fair precisely because the expensive option was never unavailable. Reconstruction is nonetheless not always informative — 7,687 of the 50,000 rows have no observable event before their observation date, leaving their historical features censored or undefined (Section 4.7.1). The regimes also happen to correspond to a runtime distinction, since a repository absent from the precomputed feature cache is scored from live metadata alone, but that is a consequence of the ablation rather than its motivation.

The design assumes that richer temporal information makes the full-history model the stronger of the two. On the aggregate held-out slice that assumption appears barely supported — the gap between the two gradient-boosted models is 0.005 ROC-AUC. Section 6.1.2 shows this reading to be an artifact of the slice composition: the value of reconstructed history becomes visible only where the label is not already determined by present quietness.

4.9 Leakage Prevention

Temporal leakage is a central threat in historical prediction tasks. The dataset builder applies the following rules to reduce leakage:

1. Observation-time features are derived only from timestamps $\leq t$.
2. Labels are derived only from timestamps in $(t, t + 12 \text{ months}]$.
3. Rows with incomplete future-window coverage remain unlabeled.
4. Live GitHub enrichment is limited to stable repository metadata such as creation time, default branch, and fork state.
5. Historical stars, forks, issues, pull requests, and releases are not taken from current GitHub API totals.
6. Seed strata such as active, dormant, and archived are used only for sampling provenance and not as labels.

These rules simulate a real prediction setting: at observation time t future activity is unknown, so information from after t is excluded from feature construction and used only for label derivation.

It is important to distinguish temporal leakage from a related but legitimate effect. Several observation-time features, such as days since the last commit, the activity-drop ratio, or days since the last release, are strongly correlated with future inactivity by construction: a repository that is already quiet at t is statistically likely to remain quiet. This is not leakage, because these features use only information available at or before t . It does mean, however, that part of the predictive signal reflects activity persistence rather than the early detection of decline. This effect motivates both the trivial recency baseline and the active-at- t subset evaluation described in Section 4.11.

4.10 Model Training

The training workflow is notebook-primary and runtime scoring is artifact-based: the scoring service expects staged model artifacts instead of retraining models during normal application use [25]. Section 5.4 describes how this separation is realized and why it makes the reported results reproducible.

The reported learned model families are Logistic Regression, XGBoost, and a feedforward neural network, each trained for the full-history and cold-start regimes. Logistic Regression is used as an interpretable linear model, XGBoost as a strong non-linear gradient-boosted tree model, and the neural network as a non-linear model that learns feature interactions through hidden layers [20]. In addition, two trivial non-learned baselines are included as references. The first ranks repositories by days since the last human commit; the second by negated annual commit volume. Both select a decision threshold on the validation slice and are evaluated on the same held-out slice as the learned models, and both are computed inside the training notebook so that every reported baseline figure originates from the run that produced the headline metrics. The baselines are necessary because the label is itself activity-related (Section 4.9): the learned models are only worth reporting if they outperform them, especially on the repositories still active at t , where Section 6.1.1 shows the learned advantage to be concentrated.

The exact model configurations and hyperparameters are documented in Appendix A.5.4; all learned models are calibrated on the validation slice before evaluation.

Training uses a time-aware split with the default ratio:

75% training, 15% validation, 10% test

Rows are ordered by observation date before splitting. The model is fitted on the earliest training slice. The validation slice is used for probability calibration and threshold selection. A histogram-based (binned) calibrator is fitted on the validation predictions, and a decision threshold is selected from the validation data. The final test slice is held out for reporting evaluation results. This design avoids a random split that would mix earlier and later observation periods and would not match the intended forecasting setting.

4.11 Evaluation Strategy

The primary evaluation is quantitative, on the time-aware held-out test slice, which is used neither for fitting nor for calibration. The reported metrics — accuracy, precision, recall, F1 score, Brier score, log loss, ROC-AUC, expected calibration error, and a combined quality score — are defined formally in Section A.7. Because the inactive base rate in the slice is not extreme (approximately 0.66), ROC-AUC is a reasonable primary ranking metric: the objection that it can present an overly optimistic picture applies to strongly imbalanced problems, where a precision-recall curve is the more informative summary [23]. Precision and recall are reported at the selected operating point, but because precision depends on the positive-class prevalence and the sampled prevalence is not the expected deployment prevalence, precision is interpreted as conditional on the sampled prevalence rather than as a deployment guarantee, and is not treated as a headline comparison metric. Calibration is assessed separately, since a probability-like risk score must match observed outcome frequencies and not merely rank correctly [24].

Two comparisons guard against inflated headline numbers. The learned models are compared against the trivial recency baseline, and they are additionally evaluated on the subset of repositories still active at t — the harder and more decision-relevant case, because predicting that an already dormant repository stays inactive is close to trivial. Reporting this subset separately prevents activity persistence (Section 4.9) from dominating the overall metrics.

This quantitative evaluation is complemented by manual case review, because inactivity risk is an operational proxy rather than a directly observable truth; such review identifies cases where the model is right for the wrong reason or where context demands caution, and it drove the feature-set revision in the first iteration. The held-out split

is a valid *internal* evaluation of temporal generalization only; its limits as evidence of generalization beyond the constructed dataset are treated as a threat to external validity in Chapter 8.

The primary deployed full-history artifact is `xgboost-full-history`, version 0.2.0, trained with `feature-set-v4-full-history`; it is the model staged for runtime scoring. Its held-out test metrics, together with those of the Logistic Regression model, the neural network (`neural-net-full-history`), and the two trivial baselines, are reported and interpreted in Chapter 6.

4.12 Methodological Limitations

This section records limitations that are intrinsic to the method and its data. Broader validity threats and their mitigation are consolidated in Chapter 8; this section is intentionally restricted to data and design limitations.

First, the method depends on the completeness of the provided GH Archive files: missing event hours leave repository histories incomplete. Second, public GitHub activity does not capture all maintenance — projects may develop elsewhere, release off GitHub, or maintain stable software with little visible activity. Third, package-to-repository mappings can be ambiguous when packages move or metadata is incomplete. Fourth, the inactivity label is a proxy for elevated maintenance risk rather than a statement of abandonment, and its thresholds (Section 4.3) are heuristic choices whose effect on prevalence is a known robustness concern.

The foundation seed also bounds scope and prevalence: the evaluation covers visible open-source repositories with a license and a minimum star count rather than all of GitHub, and the oversampling of dormant and archived repositories (Section 4.4) makes predicted probabilities conditional on the sampled prevalence, to be recalibrated before they are read as deployment-time risk estimates.

Finally, the internal test split evaluates temporal generalization within the constructed dataset but does not replace external validation on a newly sampled repository set, which is identified as future work in Chapter 9 rather than claimed as a completed result.

5 Implementation

This chapter describes how the methodological design of OSS Risk Radar was realized in software. It concentrates on the parts that bear on the research question: the overall system architecture, the leakage-controlled historical dataset builder, the two dataset-construction corrections that emerged from the design cycle, and the instrumentation that supports the reported evaluation. Engineering detail that is needed only to reproduce or operate the artifact — the language and technology choices, the dataset-builder module layout, the model configurations, the artifact format, and the scoring service, backend, frontend, and deployment — is collected in Appendix A.5 so that it does not crowd out the scientifically relevant decisions. The artifact remains reproducible and auditable, consistent with the Design Science Research framing of the thesis [11], [12].

5.1 System Architecture Overview

Three properties of the system bear on the research question. The remaining engineering detail — the component inventory and the language choices — is deferred to Appendix A.5 and Appendix A.5.1.

First, the training code and the runtime scoring service share one versioned copy of the feature definitions. A feature computed one way during training and another way at scoring time would silently invalidate every reported metric, because the model would be evaluated on one function of the data and deployed on another.

Second, training and scoring are strictly separated. Models are never trained during normal application use: the training pipeline exports versioned artifacts that are promoted into a staged bundle, and the scoring service loads only these; a missing artifact is a configuration error rather than a reason to substitute fallback behavior. This is what allows Chapter 7 to be read as evidence about the evaluated model.

Third, the unit of analysis is a single repository, scored independently. The dependency inventory to grade is supplied by an external software-composition-analysis tool such as

the OSS Review Toolkit [29], so the artifact does not resolve dependency graphs itself [25].

5.2 Historical Dataset Builder

The historical dataset builder is implemented as a Python package under `app/training/maintenance_dataset`. It transforms a repository seed and raw GH Archive event files into an inspectable, leakage-controlled training dataset, and is divided into focused modules for ingestion, event parsing, feature computation, and label computation so that each stage can be tested independently; the module layout is documented in Appendix A.5.2, and the dataset-quality report it emits in Appendix A.5.3.

The separation between observation-time and future-window computation is enforced in code. Each snapshot fixes its feature window to the year before t , a previous window to the year before that, and a label window of twelve months after t . Feature computation only consults events at or before the observation date, and label computation only consults events strictly after it. The label implementation reflects the two-of-four maintenance rule directly: it counts the number of distinct months with human commits, the number of distinct future contributors, the number of future releases or published package versions, and the number of merged pull requests, and marks a repository as maintained only if it is not archived or deleted within the horizon and at least two of these conditions are met. When the dataset-wide archive coverage horizon does not span the full label window, the row is marked with a `future_window_incomplete` signal and left unlabeled, so that missing data is never silently interpreted as inactivity. The granularity at which this coverage horizon is measured was itself the subject of a correction, discussed in Section 5.2.1.

Human-versus-bot attribution is applied during both feature and label computation. Commit and contributor counts use an actor filter that excludes automated accounts, so that bot-driven activity does not inflate maintenance evidence. This is consistent with the cautious treatment of responsiveness signals motivated in the methodology, where automated responses are known to distort interaction metrics [15].

The export step produces two outputs with different purposes. The training snapshot file follows the existing training contract and feeds the model-training pipeline. The repository feature cache stores, for each repository, the most recent observation-time feature row, with diagnostic-only fields such as `repo_archived_at_obs` removed. This cache is what later allows the runtime backend to score a previously seen repository in the richer full-history regime rather than the cold-start regime.

Two design-cycle refinements of the builder are worth recording. First, the feature set was revised after the formative review to replace raw counts that were poor point-in-time predictors (such as an absolute open-issue count, which large healthy projects also carry) with relational and trend-based variables, and to resolve a single primary package-to-repository link per repository so that mirror repositories do not make an actively developed project look inactive; the details are given in Appendix A.5.12. Second, and more consequential for the labels, is the completeness correction described next.

5.2.1 Label-Completeness Correction

A further design-cycle iteration corrected how label completeness is determined. The prediction objective, the label window $(t, t+12 \text{ months}]$, and the two-of-four maintenance rule are unchanged; the iteration concerns only which observation rows are admitted as labeled examples, and is therefore a dataset-construction correction rather than a change of the model or its target.

The methodology requires that a row be labeled only when the historical data actually covers its full future window, precisely because the absence of future activity is meaningful only when the future period is observed (Section 4.9). The initial implementation enforced this using each repository’s own last observed event as its coverage end. This proxy is biased in a way that is correlated with the outcome: a repository that simply falls quiet has, by definition, no late events, so its coverage end is early and its later observation rows were discarded as “incomplete”, even when the archive fully covered the corresponding window. The very repositories that exhibit the inactivity the model is meant to detect were thus systematically under-represented among the labeled positives. The symptom was visible in the smoke fixtures, where a synthetic late “coverage marker” event had to be injected into an otherwise inactive repository purely to keep its rows labelable.

The corrected implementation gates completeness on a dataset-wide archive coverage horizon rather than on any single repository’s last event. This horizon is the latest event observed across all repositories in the run, or an explicit cutoff supplied through the new `--coverage-end` option, which the orchestration script defaults to the latest day of complete hourly GH Archive coverage. A repository that is silent but still inside this horizon is now labeled inactive, as intended, while rows whose window genuinely extends past the available data remain unlabeled. For transparency, a row whose own last event precedes the horizon but which remains labelable is annotated with a `repo_silent_before_horizon` diagnostic signal, so that the share of rescued silent

repositories can be reported as part of the dataset-quality summary. This change aligns the implementation with the leakage-prevention rule already stated in the methodology and removes a selection effect that previously depressed the labeled inactive population.

5.3 Model Training and Evaluation Instrumentation

OSS Risk Radar implements three learned model families for both the full-history and the cold-start regimes: Logistic Regression and a feedforward neural network, both implemented from first principles so that the artifact stays dependency-light and fully inspectable, and XGBoost through its established library. All three share the same dataset, the same time-aware split, and the same calibration and evaluation code; the exact model configurations are documented in Appendix A.5.4 and the artifact format in Appendix A.5.5.

All three families produce raw probabilities that are post-processed identically. A histogram (binned) calibrator is fitted on the validation predictions — grouping them into probability bins, taking each bin’s smoothed empirical inactivity rate, and enforcing a monotonic mapping — and is stored with the artifact and re-applied at inference, so the runtime service reproduces exactly the calibration that was evaluated. At inference the calibrated probability is obtained by piecewise-linear interpolation between the populated bins rather than by a flat per-bin lookup. The distinction matters in practice: a flat lookup maps every raw score inside a bin onto the same calibrated value, which collapses distinct repositories onto identical risk scores and makes a ranked triage view degenerate. Interpolating between bin centres preserves the ordering of the raw scores while retaining the empirical calibration, and because the bin rates are monotone by construction the mapping remains monotone. A decision threshold is then selected on the calibrated validation predictions by maximizing the F1 score. The evaluation module computes the reported metrics (Section 4.11) on the held-out test slice, supported by two instruments whose mechanics are documented in Appendix A.5.13 and Appendix A.5.14: a *slice evaluation* that re-scores the held-out predictions within meaningful subgroups (popularity tier, history regime, and the active-at- t subset), so a strong majority subgroup cannot mask a weak minority one; and a *repository-disjoint split* that confines every snapshot of a repository to one partition, so the held-out set contains only unseen repositories and the optimism from repository recurrence is bounded. Both leave the serialized artifacts and headline metrics unchanged, and their per-slice and per-split figures are reported in Chapter 6.

5.4 Reproducibility of the Evaluated Artifact

Every figure reported in this thesis must be traceable to a fixed model, fitted on a fixed dataset, by a procedure that can be re-executed. Three mechanisms establish that.

Training is notebook-primary: `notebooks/oss-maintenance-training.ipynb` is both the visible thesis workflow and the artifact export boundary, and `npm run ml:train` executes it non-interactively (Appendix A.5.6), so the reported results come from the same code path a reader would run rather than from a separate analysis script. Each run is then serialized to a self-describing artifact recording the ordered feature names, fitted parameters, calibration bins, decision threshold, feature-set version, and the hash of the dataset it was fitted on (Appendix A.5.5); that hash is stated in Appendix A.8, so a rebuilt dataset can be verified against the one evaluated here.

Finally, artifacts are not deployed automatically. A staging command promotes a candidate bundle only if it passes a regression comparison against the currently deployed baseline: for each required model, ROC-AUC must not drop and the Brier score must not increase beyond a configured tolerance (Appendix A.5.7). The model that produced the demonstrator output in Chapter 7 is therefore verifiably the model evaluated in Chapter 6.

The runtime components that consume these artifacts — the scoring service (Appendix A.5.8), the backend (Appendix A.5.9), the frontend (Appendix A.5.10), and the deployment (Appendix A.5.11) — are engineering context rather than part of the argument, and no result in this thesis depends on them.

The next chapter reports and discusses the empirical results obtained with this implementation.

6 Results and Discussion

This chapter reports and interprets the empirical results of the trained artifacts: predictive performance against non-learned baselines, calibration behaviour, feature attribution and error analysis, and an ablation of the human-actor filter.

The central result is stated up front, because it shapes how the rest of the chapter should be read. On the held-out slice as a whole, a single observable signal — the number of days since the last human commit — already ranks inactivity almost as well as a sixty-feature gradient-boosted model. The learned models are nevertheless worth their complexity, but not for the reason an aggregate metric would suggest: their advantage concentrates on the repositories that are still active at the observation time, which is precisely the population for which an early-warning signal has any decision value at all.

6.1 Predictive Performance

Table 6.1 reports held-out test metrics for the three learned model families in both feature regimes, together with two non-learned baselines. All learned models were trained on the same time-aware split (75/15/10; $n_{\text{test}} = 5,000$ labelled observations, inactive base rate 0.661) and calibrated on the validation slice with the same histogram calibrator, so the comparison isolates the effect of model family rather than data or calibration differences. The combined quality score summarizes ranking skill (ROC-AUC) and calibration (Brier) into a single comparison figure and is not intended as a standalone proof. All reported metrics are defined formally in Section A.7.

Among the learned models the ordering is stable across both regimes. The gradient-boosted trees (`xgboost-full-history`) achieve the strongest ranking and probabilistic accuracy (ROC-AUC 0.958, Brier 0.078), ahead of the multilayer perceptron (0.956 / 0.081) and logistic regression (0.949 / 0.088). The neural network was included as a deliberate test of whether a higher-capacity non-linear model is warranted for this tabular feature space. It does not improve on the gradient-boosted trees on ranking

Table 6.1: Held-out test metrics by model family and feature regime, with the two trivial baselines. Higher is better for ROC-AUC, F1, and quality; lower is better for Brier and ECE. The best value within each regime is shown in bold. The recency baseline is defined on the 4,556 test rows that have an observable commit history; see the text for the like-for-like comparison.

Model	ROC-AUC	Brier	ECE	F1	Quality
logistic-regression-full-history	0.949	0.088	0.047	0.911	0.782
xgboost-full-history	0.958	0.078	0.035	0.916	0.810
neural-net-full-history	0.956	0.081	0.028	0.913	0.803
logistic-regression-cold-start	0.932	0.100	0.044	0.897	0.740
xgboost-cold-start	0.953	0.082	0.033	0.909	0.798
neural-net-cold-start	0.951	0.084	0.027	0.908	0.791
recency-baseline (days since commit)	0.957*	—	—	0.918	—
commit-volume-baseline (−commits_365d)	0.897	—	—	0.904	—

or Brier score in either regime, while adding training cost and weaker interpretability; its one advantage is the lowest expected calibration error. This is consistent with the broader finding that tree-based models remain a strong default on moderate-sized tabular problems [30]. Accordingly, logistic regression is retained for its transparent coefficient attribution and XGBoost as the deployed runtime scorer, while the neural network is reported as a tested-and-rejected alternative rather than promoted into the scoring ensemble.

The cold-start regime is only marginally weaker than the full-history regime for the non-linear models: `xgboost-cold-start` reaches ROC-AUC 0.953 against 0.958 for its full-history counterpart, despite using 17 rather than 60 features and, in particular, despite lacking the commit-window features that dominate the full-history model’s gain (Section 6.3.1). Only the linear model degrades appreciably (0.932 against 0.949). The regimes are therefore not as far apart as the feature counts suggest, because the cold-start vector retains `last_push_age_days`, a recency signal that is nearly as informative as the reconstructed commit history. This appears to qualify the expectation stated in Section 4.8 that the full-history model would be clearly superior: for the deployed tree model it is better, but only by 0.005 ROC-AUC. That reading does not survive a closer look. Section 6.1.2 shows the aggregate gap to be an artifact of the slice composition, and the regimes to separate by 0.030 ROC-AUC once the already-quiet majority is removed.

6.1.1 Comparison Against Non-Learned Baselines

Because the inactivity label is itself activity-derived, a learned model is only worth reporting if it improves on a trivial rule built from a single observable signal (Section 4.10). Two such rules are evaluated on the same held-out slice: ranking by days since the last human commit, and ranking by negated annual commit volume.

The result is sobering, and it is the most important quantitative finding of this thesis. The recency rule attains ROC-AUC 0.957 and an F1 score of 0.918, which is not merely competitive with the deployed `xgboost-full-history` model but nominally exceeds its F1 of 0.916. The comparison is not quite like-for-like: `days_since_last_commit` is undefined for the 444 test rows whose repositories have no observable record before the observation date (Section 4.7.1), so the baseline is scored on 4,556 rows while the model is scored on all 5,000. Re-scoring the deployed model on exactly those 4,556 rows raises it to ROC-AUC 0.974 against the baseline’s 0.957. The model therefore does outrank the trivial rule, but by 0.017 ROC-AUC rather than by the wide margin that an aggregate comparison against a weaker baseline would suggest. The commit-volume rule is clearly worse in aggregate (0.897).

Two conclusions follow. First, the aggregate held-out metric is a poor instrument for judging this artifact: most of the achievable ranking skill on a slice whose inactive base rate is 0.661 is already contained in one recency signal, and a sixty-feature model recovers only a modest increment on top of it. Reporting the headline ROC-AUC alone would overstate what the learned model contributes. Second, this is exactly the situation the active-at- t evaluation was designed to expose (Section 4.11). Among repositories that are already quiet at t , predicting continued inactivity is close to trivial, and both the model and the recency rule do it well; the aggregate metric is dominated by that easy majority. The decision-relevant question is what happens on the repositories that are still active, and there the picture separates sharply, as Section 6.3.3 shows.

6.1.2 How Much Is Reconstructed History Worth?

The same instrument answers the feature-availability question posed in Section 4.8. Comparing the two gradient-boosted models isolates the contribution of the 43 reconstructed historical features, since the cold-start model differs from the full-history model only by their absence. Table 6.2 reports every arm on identical rows: where the recency signal is undefined the row is dropped from *all* arms rather than imputed for the models alone.

Table 6.2: Held-out ROC-AUC by feature-set size, on identical row sets.

Subset	n	Full-history (60)	Cold-start (17)	Best trivial rule (1)
Recency-defined	4,556	0.974	0.970	0.958
active@ t	1,794	0.905	0.875	0.844

In aggregate, reconstructed history is worth 0.005 ROC-AUC, and the 17 snapshot features are worth 0.013 over the single best trivial signal. Read alone, those numbers say the GH Archive pipeline was not worth building. On the active-at- t subset the comparison inverts: history is worth 0.030, six times its aggregate value, and the snapshot features 0.083. The ordering of the three feature sets — one signal, seventeen, sixty — is strictly monotone where early warning matters and almost flat where it does not.

This is the third independent appearance of one property of the data. The learned model beats the trivial rule by 0.017 in aggregate against 0.061 on active-at- t ; reconstructed history by 0.005 against 0.030; the snapshot feature set by 0.013 against 0.083. Each aggregate comparison is compressed by the same cause: a slice in which two thirds of repositories are already quiet rewards a predictor for restating the present, so any additional information — more features, more history, a learned decision boundary — is visible only once that easy majority is removed. The aggregate ROC-AUC of this dataset measures activity persistence more than predictive skill, and no design decision in this thesis should be judged against it.

6.2 Calibration and Threshold Selection

Because the score is intended to be read as a probability rather than only as a ranking, calibration is evaluated alongside discrimination. Every model’s raw outputs are mapped through a histogram calibrator fitted on the validation slice and never on the test slice, so the reported calibration reflects held-out behaviour. For the deployed `xgboost-full-history` model this yields a Brier score of 0.078, a log loss of 0.246, and an expected calibration error of 0.035 at the selected decision threshold of 0.51, where the operating point attains precision 0.90 and recall 0.93. Among the full-history models the neural network attains the lowest expected calibration error (0.028) and logistic regression the highest (0.047), so the ranking by calibration does not follow the ranking by discrimination.

The reliability of the calibrated probabilities is well behaved: the empirical inactiv-

ity rate rises monotonically across the ten calibration bins, from 0.032 in the lowest predicted-probability band to 0.988 in the highest, so higher scores do correspond to higher observed inactivity frequencies rather than only to a higher rank. Two caveats apply. First, the two extreme bands hold 65.9% of the validation mass, so mid-range probabilities are supported by comparatively few samples and are correspondingly less certain; the per-prediction confidence surfaced by the demonstrator (Chapter 7) reflects exactly this bin-level support. Second, calibration is conditional on the sampled prevalence rather than a natural deployment prevalence (Section 4.12), so the absolute probabilities would require recalibration before being read as live population probabilities.

6.3 Feature Importance and Error Analysis

6.3.1 Feature Attribution

Feature attribution is read from the logistic-regression model, which is retained precisely for its transparent coefficients (Section 6.1). Because the inputs are standardized, the coefficient magnitudes are directly comparable, and their signs indicate the direction of the association with inactivity. The strongest inactivity-*lowering* signal is the breadth of sustained development (`active_commit_months_365d`, standardized coefficient -0.79), followed by recent maintainer integration (`has_recent_pr_merge_flag`, -0.39) and the presence of any issue activity at all (`has_issue_activity_365d`, -0.26). The strongest inactivity-*raising* signals are the two staleness measures, days since the last push and days since the last commit (each $+0.56$). These directions match the intended semantics of the features (Appendix A.2).

Two coefficients resist that reading. A larger open-issue backlog is associated with *lower* inactivity (`open_issues_log1p`, -0.41), and a larger star count with *higher* inactivity (`stars_log1p`, $+0.25$). Both are plausibly confounded rather than causal: an open backlog presupposes a user base that files issues, and popularity is correlated with project age within the sampled frame. These coefficients are therefore used to explain *direction* for a given repository, while ranking performance is carried by the gradient-boosted model.

These attributions are only interpretable because of the encoding correction of Section 4.7.1. While an undefined response latency was stored as zero, the responsiveness medians ranked among the strongest inactivity signals, inviting the reading that slow-responding projects fail; in fact a zero latency marked projects with *no* issues or pull

requests at all. After the correction those medians no longer appear among the leading coefficients, and the absence of activity is carried explicitly by `has_issue_activity_365d` and `has_pr_activity_365d`.

The gradient-boosted model exposes a complementary, non-linear view through its gain-based feature importances (normalized to sum to one over the 42 of 60 features that receive at least one split). Its ranking is dominated by recent commit volume: `commits_365d` (0.42) and `commits_90d` (0.20) together account for over three-fifths of the total gain, followed by `active_commit_months_365d` (0.08) and `has_recent_pr_merge_flag` (0.07); `last_push_age_days`, `opened_prs_90d`, `days_since_last_commit`, and `merged_prs_90d` contribute the next tier. The two views agree on the substantive story — sustained recent development activity is the dominant signal — while differing in emphasis: the linear model leans on the breadth of active commit months, whereas the gradient-boosted model concentrates gain on the raw recent-commit-volume windows. That concentration also explains the aggregate near-parity of the cold-start model observed in Section 6.1: the cold-start vector omits both commit windows, yet `last_push_age_days` substitutes for them at little cost *on a slice where most repositories are already quiet*. Where they are not, the substitution is a poorer one (Section 6.1.2).

6.3.2 Robustness to Repository Recurrence

Under the repository-disjoint split (Section A.5.14), where every snapshot of a repository is confined to one partition, held-out ROC-AUC decreases from 0.958 to 0.950 and the Brier score rises from 0.078 to 0.079. The small gap bounds the optimism attributable to a repository appearing in both training and test slices and indicates that the discrimination generalizes to unseen repositories rather than resting on repository memorization. This check matters more than it did previously: because the labelled dataset now spans ten equally sized quarterly cohorts, the chronological test slice consists entirely of the final observation date, so every test repository also appears in training at an earlier date.

6.3.3 Slice Evaluation

A single aggregate metric can average a strong subgroup with a weak one. The held-out predictions of the deployed `xgboost-full-history` model are therefore re-scored within subgroups that vary in ways relevant to deployment; the full per-subgroup table is given in Appendix A.10 (Table A.10). Three findings follow.

First, and most importantly for the activity-persistence threat (Section 8.2), the model separates outcomes well *among repositories that are active at the observation time*: on the active-at- t subset ($n = 1,794$, inactive rate only 0.225) it attains ROC-AUC 0.905. On this same subset the recency baseline reaches only 0.792 and the commit-volume baseline 0.844. The learned model therefore holds a margin of 0.061 over the best trivial rule exactly where early-warning skill matters, against a margin of 0.017 in aggregate (Section 6.1.1). This is the central justification for the artifact: its value is not that it ranks a mixed population better than a recency rule, but that it anticipates decline in repositories that still look healthy. The operating point on this subset is nevertheless conservative — recall 0.49 at precision 0.74 — so the model flags fewer than half of the repositories that will fall quiet, and the score is better used as a ranking for triage than as an automatic decision.

Second, the aggregate ROC-AUC masks a much weaker low-popularity subgroup, ranked at only 0.846 against 0.972 and 0.964 for high- and medium-popularity repositories (Table A.10); these projects deserve more caution and are exactly where the per-prediction confidence should be low, though “low popularity” is relative to a seed frame that already requires at least 100 stars. Third, repositories with reconstructable history are ranked no higher than cold-start-like ones (0.921 against 0.929), but that ordering is confounded by the far higher inactive rate of the cold-start-like subgroup (0.924 against 0.350) and should not be read as evidence that history is unhelpful; it reinforces only that the two regimes are closer in skill than their feature counts suggest (Section 6.1).

6.3.4 Error Analysis

The comparison against explicit trivial baselines is now part of the reported results (Section 6.1.1 and the slice analysis above); the systematic formative case review that motivated the feature-set revision (Section 4.2) remains a complementary qualitative analysis and is noted as a remaining item in Chapter 9. The quantitative results identify two residual weaknesses. The dominant one is low-visibility repositories with sparse public signals, which the confidence layer is designed to flag rather than hide. The second is recall on the active-at- t subset: the model ranks impending decline well but, at the F1-selected threshold, commits to flagging only about half of it. Both weaknesses argue for using the score as a prioritization signal rather than as a gate.

6.4 Ablation: The Human-Actor Filter

Commit- and contributor-based features and the maintenance label are computed from human actors only; automated (bot) accounts are excluded (Section 4.7). Because this design choice could plausibly be omitted, it is tested by rebuilding the full 5,000-repository dataset with the filter disabled through a single build flag, every other setting identical. The ablation shows three effects, reported in full in Appendix A.10.1. First, disabling the filter corrupts the label one-directionally: 357 of 50,000 snapshots (0.71%) flip from inactive to active and none the other way, because a bot commit or merge in the future window can satisfy the maintenance rule and mask an otherwise dormant repository. Second, it inflates the observation-time activity features on a bot-heavy minority of repositories (for instance `commits_365d` changes on 15.7% of snapshots). Third, its predictive cost is small but consistent: trained on contaminated features and evaluated against the clean label, every model family loses 0.0006–0.0011 ROC-AUC (Table A.11).

The filter’s justification is therefore construct validity rather than discrimination: its predictive cost when omitted is minor, but it removes a one-directional label bias — the masking of inactivity — that a headline metric computed on the corrupted data would not reveal. The human-actor filter is retained.

7 Practical Demonstrator

Chapter 6 evaluates the model on a held-out slice drawn from the same seed distribution it was trained on. This chapter asks a different and harder question: what does the model do when applied to repositories it was never sampled to see? Answering it serves the demonstration phase of the Design Science Research process [11], [12], but its scientific value here is as an error analysis. An out-of-distribution application exposes failure modes that a held-out slice cannot, because the held-out slice inherits the seed’s composition and therefore its blind spots.

The input is the dependency set of a real project, scored through the deployed artifact. Because the staging gate guarantees that the deployed model is the evaluated model (Section 5.4), the outputs below are evidence about the same `xgboost-full-history` artifact (feature set `feature-set-v4-full-history`) whose metrics Chapter 6 reports, and not about a separately configured system. All figures are real outputs and reflect the public repository state on 2026-07-09, the date of the analysis run.

7.1 Example Repository Set

Consistent with the unit of analysis fixed in Section 5.1, each repository is scored independently and the dependency inventory is taken as an external input produced by software-composition-analysis tooling [29].

For this demonstration the input is the set of dependency repositories of `ArchipelagoMW/Archipelago`, a large and actively developed open-source multi-world game randomizer. Its declared Python dependencies correspond to 30 source repositories, ranging from widely used infrastructure libraries (for example `flask`, `jinja2`, `werkzeug`) to smaller, more specialized packages. This project was chosen because that mixture is expected to produce a non-trivial triage view rather than a uniformly low-risk one.

Each repository is scored through the deployed analysis workflow, which enriches live repository metadata and applies the staged model ensemble. The two prediction regimes of Section 4.8 appear directly in this run: one repository (`flask`) is present in the staged

repository feature cache and is therefore scored in the richer full-history regime, while the remaining 29 are absent from the foundation cache and fall back to the cold-start regime. This distribution is itself an honest illustration of the cold-start coverage limitation highlighted in Chapter 9: an arbitrary project’s dependency repositories are mostly outside the precomputed foundation seed. The aggregate held-out gap between the two regimes is small, but Section 6.1.2 shows that this understates the cost of scoring without reconstructed history: on repositories still active at the observation time — which is what most of the thirty are — the full-history model outranks the cold-start one by 0.030 ROC-AUC. Twenty-nine of these thirty scores are therefore drawn from the weaker of the two regimes, on precisely the population where the difference is largest. A second caveat compounds it: the cold-start metrics are measured on seed-distribution repositories while the model is applied here to arbitrary submissions.

7.2 Risk Ranking Output and Interpretation Guidance

The scoring service produces, for each repository, the inactivity-risk score (0–100), the discrete action level (`monitor`, `review`, or `replace_candidate`), a confidence value derived from signal completeness and prediction decisiveness, and the prediction regime. Of the 30 scored repositories, the risk buckets are two critical, three high, eight medium, and 17 low; by action level, two repositories are raised to `replace_candidate`, four to `review`, and the remaining 24 to `monitor`. The ranking therefore concentrates analyst attention on six of thirty repositories rather than diluting it across the whole set. Table 7.1 shows the flagged repositories together with the full-history item for contrast; the complete 30-repository ranking is given in Appendix A.9. Every one of the 30 scores is distinct, which reflects the interpolated calibration mapping of Section 5.3 rather than the flat per-bin lookup it replaced.

The highest-ranked repository. The item the ranking elevates most strongly is `bsdifff4` (`ilanschnell/bsdifff4`), a small binary-diff library, with a risk score of 92.1. The score reflects a genuinely quiet maintenance profile — a low-popularity, single-maintainer repository with no detected release cadence — which is exactly the kind of dependency a supply-chain reviewer would want surfaced, and its moderate confidence of 0.69 and cold-start caveats present the flag as a prompt for manual review rather than a definitive judgement (the underlying explanation factors are given in Appendix A.9).

Table 7.1: Flagged dependency repositories of ArchipelagoMW/Archipelago (the two `replace_candidate` items and the four `review` items), with the single full-history repository shown for contrast. Risk is the 0–100 inactivity-risk score; “Conf.” is the per-prediction confidence; “Regime” is full-history (FH) or cold-start (CS). The complete 30-repository ranking is in Appendix A.9.

Package	Source repository	Risk	Action	Conf.	Regime
bsdiff4	ilanschnell/bsdiff4	92.1	<code>replace_candidate</code>	0.69	CS
setproctitle	dvarrazzo/py-setproctitle	80.4	<code>replace_candidate</code>	0.61	CS
jinja2	pallets/jinja	77.1	<code>review</code>	0.77	CS
cymem	explosion/cymem	74.9	<code>review</code>	0.75	CS
markupsafe	pallets/markupsafe	74.2	<code>review</code>	0.75	CS
pymem	srounet/Pymem	54.3	<code>review</code>	0.47	CS
flask	pallets/flask	9.5	<code>monitor</code>	0.91	FH

Two instructive false positives. The ranking also raises `jinja2` (77.1) and `markupsafe` (74.2) to `review`. Both are Pallets projects: widely used, professionally maintained, and in no meaningful sense at risk of abandonment. Neither elevation can be blamed on missing data — both were scored with 100% signal completeness — so these are genuine model errors rather than artifacts of imputation, and they are reported here rather than omitted. The explanation is the failure mode that Section 2.3 anticipates: a mature, feature-complete library that has reached a stable API exhibits precisely the observable profile of a project winding down — infrequent commits, long gaps between releases, a small active contributor set — because low activity and low *need* for activity are indistinguishable in public metadata. The model was trained on a label that defines inactivity as the absence of observable maintenance signals (Section 4.3), so it is behaving exactly as specified; what it cannot express is that for `jinja2` the absence of churn is a sign of maturity. This is the strongest practical argument in this thesis for treating the score as a prioritization signal rather than a verdict, and for the human review step that the action levels are designed to trigger.

A low-risk repository for contrast. At the opposite end, `flask` (`pallets/flask`) receives a risk score of 9.5 with a confidence of 0.91. It is the one repository scored in the full-history regime, with complete signal coverage and no imputation, and its two ensemble members agree closely (5.4% and 13.5% inactivity risk). That `flask` is ranked low while its sibling projects `jinja2` and `markupsafe` are flagged is instructive in itself: the three are maintained by the same organization under the same release practices, and the difference in their scores tracks the availability of reconstructed history rather than any difference in maintenance. This contrast between a moderate-confidence cold-start flag and a high-confidence full-history clearance illustrates why the confidence value,

not only the risk score, is part of the triage output — and it quantifies the cold-start coverage gap more sharply than the held-out metrics do.

Interpretation guidance follows the rest of the thesis. The score is a prioritization signal, not a trust or security verdict: a low score does not certify a repository as secure, and an elevated score indicates thin public maintenance signals rather than a known defect. Low confidence or a large number of imputed signals should reduce the decisiveness of any action, and cold-start scores in particular are conditional on the reduced feature set and on the sampled prevalence to which the model is calibrated (Section 4.12), so they warrant recalibration before being read as absolute deployment probabilities. Used this way, the ranked view converts a flat list of 30 repositories into six candidates for review, of which at least two — on the evidence above — a human reviewer would quickly clear. That is a useful reduction of analyst effort, and an honest statement of the artifact’s precision at this operating point.

8 Threats to Validity

This chapter consolidates the threats to validity for the thesis and the mitigations applied. It is the single home for validity discussion; the methodology chapter (Section 4.12) is restricted to intrinsic data and design limitations, which are referenced here rather than repeated. Threats are organized along the standard categories of construct, internal, external, and conclusion validity.

8.1 Construct Validity

Construct validity concerns whether the measured quantities represent the intended concepts. The central construct is repository inactivity, which is operationalized as the inverse of a multi-signal maintenance rule over the future window $(t, t + 12 \text{ months}]$ (Section 4.3). This is a proxy: a repository can be stable and low-activity without being abandoned, and visible activity does not capture development that moves to other platforms or release channels. The two-of-four maintenance thresholds are heuristic design choices, and the merged-pull-request clause is a proxy for broader maintainer interaction. These choices are mitigated by requiring sustained rather than incidental activity and by interpreting the score as a triage signal rather than a verdict, but they remain a construct-validity threat that a label sensitivity analysis (Chapter 9) would further address. A related construct decision — excluding automated (bot) actors from the activity features and the label — is validated directly by the ablation in Section 6.4: disabling the filter flips 0.71% of labels from inactive to active and never the reverse, confirming that the filter removes a one-directional bias that masks inactivity rather than merely improving model fit.

8.2 Internal Validity

Internal validity concerns whether the reported predictive relationship is genuine rather than an artifact of the experimental setup. The main threat is temporal leakage,

mitigated by the leakage-prevention rules in Section 4.9: observation-time features use only timestamps $\leq t$, labels use only the future window, rows with incomplete future coverage remain unlabeled, and live enrichment is restricted to stable repository metadata. A distinct, non-leakage threat is activity persistence: features such as days since last commit are predictive of near-term inactivity by construction, so part of the discrimination reflects already-quiet repositories staying quiet. This is mitigated by two complementary checks, and the results show that the threat is real rather than hypothetical. Comparison against explicit trivial baselines establishes that most of the aggregate skill *is* activity persistence: a single recency signal nearly matches the model on the full held-out slice (Section 6.1.1). The decisive check is therefore the active-at- t subset evaluation, which restricts the comparison to repositories still active at the observation time; there the model outranks the best trivial rule by 0.061 ROC-AUC (Section 6.3.3). The artifact’s predictive contribution is thus established on the subset where persistence cannot explain it, and is deliberately not claimed from the aggregate metric.

A second internal-validity concern is repository recurrence across the chronological split. Because observations are quarterly, a repository contributes several snapshots with overlapping label windows, and the chronological split can place earlier snapshots of a repository in training and later ones in the held-out set; given the autocorrelation between a repository’s snapshots, this makes the reported metric optimistic relative to performance on previously unseen repositories. The concern is sharpened by the shape of the reported split: because the labeled cohorts are equally sized, the chronological test partition consists entirely of the final observation date, so *every* test repository also occurs in training at an earlier date (Section 4.6). This threat is addressed by additionally evaluating the model under a repository-disjoint split, in which every snapshot of a repository is confined to a single partition (Section A.5.14). The held-out ROC-AUC decreases only marginally under that split (Section 6.3.2), which bounds the optimism attributable to repository recurrence and indicates that the discrimination generalizes to unseen repositories rather than resting on repository memorization.

8.3 External Validity

External validity concerns generalization beyond the studied sample. Two threats apply. First, the foundation seed targets repositories with a license object and a minimum of 100 stars, so results should not be generalized to very small, private, newly created, or low-visibility repositories; the “low popularity” subgroup of Section 6.3.3 is low only relative to that frame. Second, the seed is designed to oversample dormant and archived

repositories, so the inactive base rate in the test slice does not reflect the natural OSS inactivity rate; predicted probabilities and precision are conditional on this sampled prevalence and would require recalibration for a deployment population (Section 4.12). The held-out split evaluates temporal generalization within the constructed dataset but not generalization to a freshly sampled repository set; external validation on out-of-foundation repositories is identified as future work (Chapter 9).

8.4 Conclusion Validity

Conclusion validity concerns whether the conclusions drawn from the metrics are warranted. The evaluation uses a single time-aware held-out split rather than repeated resampling, so reported metrics are point estimates without confidence intervals. Because prevalence is sampled rather than natural, precision-dependent conclusions are reported as conditional on the sampled prevalence, and ROC-AUC is used as the primary ranking metric with precision-recall analysis as a complement (Section 4.11). Qualitative case review is used as supporting, not confirmatory, evidence and is interpreted as formative.

9 Conclusion and Outlook

This thesis asked to what extent publicly observable repository health and maintenance indicators can predict 12-month OSS dependency inactivity, and how the resulting inactivity-risk score can be combined with parallel security-practice signals to support dependency triage. The empirical results support a qualified positive answer, and the qualification is essential. On a time-aware held-out split, the deployed full-history gradient-boosted model ranks inactivity with a ROC-AUC of 0.958 and is well calibrated (Brier 0.078, expected calibration error 0.035). The skill largely persists under a repository-disjoint split (0.950), so it generalizes to unseen repositories rather than memorizing them.

The aggregate figure, however, overstates what the model contributes. A trivial rule that ranks repositories by days since their last human commit attains ROC-AUC 0.957 on the same held-out rows, against 0.974 for the model: most of the achievable aggregate skill is contained in a single recency signal, because a slice whose inactive rate is 0.661 is dominated by repositories that were already quiet at the observation time, for which predicting continued inactivity is close to trivial. The learned model’s genuine contribution appears where an early-warning signal has decision value: among repositories that are still active at the observation time (inactive rate 0.225), it attains ROC-AUC 0.905 against 0.844 for the best trivial rule — a margin of 0.061, against 0.017 in aggregate. The answer to the research question is therefore that public repository signals predict 12-month inactivity well, that most of that predictive content is recency, and that a learned combination of many signals adds a modest but real capacity to anticipate decline in repositories that still look healthy.

At the same time, the model is markedly weaker on low-visibility repositories (ROC-AUC 0.846), its recall on the decision-relevant active-at- t subset is only 0.49 at the selected threshold, and its probabilities are calibrated to a sampled rather than a natural prevalence. The score is therefore presented as a calibrated triage aid with an explicit per-prediction confidence that flags exactly these weak cases, not as a verdict on abandonment. Security-practice signals remain complementary triage context and are excluded from the inactivity model (Section 3.3), so no claim is made that they improve

inactivity prediction. Applied per repository to a project’s dependency repositories, these scores produce the ranked, evidence-backed triage view demonstrated in Chapter 7, which is the intended practical contribution.

9.1 Outlook and Future Work

Several extensions are deliberately scoped out of the completed method and are recorded here as future work rather than as claimed results.

- **External validation.** The reported evaluation uses a time-aware held-out split within the constructed foundation dataset (Section 4.11). A stronger robustness check would re-sample a fresh set of repositories that were not part of the foundation seed and evaluate the trained artifacts on that independent set. This would test generalization beyond the sampled prevalence and seed strata.
- **Systematic qualitative error analysis.** The quantitative checks are complete: the active-at- t slice isolates predictive skill from activity persistence (Section 6.3.3), and the trivial baselines are reported and outperformed on that slice (Section 6.1.1). What remains is a systematic, larger-scale qualitative case review — beyond the formative examples that motivated the feature-set revision — to establish where the model is right for the wrong reason. This is complementary evidence that a purely metric-based evaluation cannot supply.
- **Additional model families.** Beyond the covered families, further extensions such as more expressive deep-learning architectures or sequence models over the repository event history are left as future work.
- **Ensemble evaluation.** The runtime averages the staged models of a regime into an ensemble profile (Section 6.1), but the reported metrics are for the individual artifacts. Benchmarking whether the ensemble improves on its strongest member, and whether the model-agreement spread is a useful uncertainty signal, is left as future work.
- **Prevalence-aware calibration.** Because the seed oversamples dormant and archived repositories, predicted probabilities are calibrated to the sampled prevalence (Section 4.12). Recalibrating to an estimated deployment prevalence would make the scores directly usable as live triage probabilities.
- **Label sensitivity analysis.** The maintenance label uses heuristic thresholds (Section 4.3). A systematic sensitivity analysis over these thresholds would

quantify how strongly the results depend on the specific label definition.

- **Cold-start coverage and on-demand history reconstruction.** Repositories outside the foundation seed are scored by the cold-start model, which uses only features derivable from a single live snapshot and therefore omits the strongest historical signals (most notably the distribution of commit activity across the year). Two observations make this a priority for future work. First, the held-out cold-start metrics are measured on seed-distribution repositories but applied to arbitrary submissions, so the deployed cold-start performance is likely optimistic; the per-prediction confidence already flags such out-of-distribution cases rather than hiding them. Second, a submitted repository’s recent history can in principle be reconstructed on demand from the GitHub REST API (for example, weekly commit activity and per-contributor statistics) to recover several full-history features. This was deliberately not implemented, because the live API cannot supply the full feature set: package-registry features (package age, dependency count, published versions, popularity tier) are not exposed by the repository API, per-issue first-response latency requires prohibitively many timeline requests, and the year-over-year change features require a second year of history. Imputing this missing tail into the full-history model would itself introduce a distribution shift. A cleaner extension is therefore a third *warm-start* model trained on exactly the subset of features that can be reconstructed reliably from the live API, validated for parity against the precomputed seed feature cache and supported by a rate-limit-aware request cache. This is left as future work because of the additional engineering and validation effort it requires.

Bibliography

- [1] Open Source Initiative, *The open source definition*, Accessed: 2026-02. [Online]. Available: <https://opensource.org/osd>
- [2] M. Souppaya, K. Scarfone, and D. Dodson, “Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities,” National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-218, Feb. 2022. DOI: 10.6028/NIST.SP.800-218
- [3] SLSA Community, *Slsa specification v1.2*, Nov. 2025. [Online]. Available: <https://slsa.dev/spec/v1.2/>
- [4] C. Miller, M. Jahanshahi, A. Mockus, B. Vasilescu, and C. Kästner, “Understanding the response to open-source dependency abandonment in the npm ecosystem,” in *Proc. IEEE/ACM 47th International Conference on Software Engineering*, 2025, pp. 2355–2367. DOI: 10.1109/ICSE55347.2025.00004
- [5] C. Miller, C. Kästner, and B. Vasilescu, ““we feel like we’re winging it:” a study on navigating open-source dependency abandonment,” in *Proc. ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 1281–1293. DOI: 10.1145/3611643.3616293
- [6] G. A. A. Prana et al., “Out of sight, out of mind? how vulnerable dependencies affect open-source projects,” *Empirical Software Engineering*, vol. 26, no. 4, 2021. DOI: 10.1007/s10664-021-09959-3
- [7] npm, Inc., *Kik, left-pad, and npm*, Mar. 2016. [Online]. Available: <https://blog.npmjs.org/post/141577284765/kik-left-pad-and-npm>
- [8] Equifax, *Equifax releases details on cybersecurity incident*, Sep. 2017. [Online]. Available: <https://investor.equifax.com/news-events/press-releases/detail/237/equifax-releases-details-on-cybersecurity-incident>
- [9] Federal Trade Commission, *Equifax to pay \$575 million as part of settlement with ftc, cfpb, and states related to 2017 data breach*, Jul. 2019. [Online]. Available: <https://www.ftc.gov/news-events/news/press-releases/2019/07/equifax-pay-575-million-part-settlement-ftc-cfpb-states-related-2017-data-breach>

- [10] Open Source Security Foundation and OpenJS Foundation, *Open source security and openjs foundations issue alert for social engineering takeovers of open source projects*, Apr. 2024. [Online]. Available: <https://www.linuxfoundation.org/blog/openssf-openjs-alert-for-social-engineering-takeovers-of-open-source-projects>
- [11] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [12] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A design science research methodology for information systems research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. DOI: 10.2753/MIS0742-1222240302
- [13] GitHub, *Rest api endpoints for licenses*, Accessed: 2026-02. [Online]. Available: <https://docs.github.com/en/rest/licenses/licenses>
- [14] D. Robinson, K. Enns, N. Koulecar, and M. Sihag, “Two approaches to survival analysis of open source python projects,” *arXiv preprint*, 2022. arXiv: 2203.08320 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2203.08320>
- [15] K. A. Hasan, M. Macedo, Y. Tian, B. Adams, and S. H. H. Ding, “Understanding the time to first response in github pull requests,” in *Proc. IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, 2023.
- [16] Y. Yehudi, C. Goble, and C. Jay, “Individual context-free online community health indicators fail to identify open source software sustainability,” *arXiv preprint*, 2023. arXiv: 2309.12120 [cs.SE]. [Online]. Available: <https://arxiv.org/abs/2309.12120>
- [17] CHAOSS Community, *Metrics model: Starter project health*, Accessed: 2026-02. [Online]. Available: <https://chaoss.community/kb/metrics-model-starter-project-health/>
- [18] N. Zahan, P. Kanakiya, B. Hambleton, S. Shohanuzzaman, and L. Williams, “Openssf scorecard: On the path toward ecosystem-wide automated security metrics,” *arXiv preprint*, 2022. arXiv: 2208.03412 [cs.CR]. [Online]. Available: <https://arxiv.org/abs/2208.03412>
- [19] S. P. Goggins, M. Germonprez, and K. Lombard, “Making open source project health transparent,” *Computer*, vol. 54, no. 8, pp. 104–111, 2021. DOI: 10.1109/MC.2021.3084015
- [20] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: With Applications in R*, 2nd ed. Springer, 2021. DOI: 10.1007/978-1-0716-1418-1

- [21] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324
- [22] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. DOI: 10.1214/aos/1013203451
- [23] T. Saito and M. Rehmsmeier, “The precision-recall plot is more informative than the roc plot when evaluating binary classifiers on imbalanced datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. DOI: 10.1371/journal.pone.0118432
- [24] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proc. 22nd International Conference on Machine Learning*, 2005, pp. 625–632. DOI: 10.1145/1102351.1102430
- [25] E. Krause, *Oss risk radar*, Source repository, pinned at commit 680b7b4. Accessed: 2026-06, 2026. [Online]. Available: <https://github.com/esuarkeN/oss-risk-radar/tree/680b7b4e747b254cc358b175722265b6d282a639>
- [26] E. Krause, *Historical dataset builder*, Pinned at commit 680b7b4. Accessed: 2026-06, 2026. [Online]. Available: <https://github.com/esuarkeN/oss-risk-radar/blob/680b7b4e747b254cc358b175722265b6d282a639/docs/methodology/historical-dataset-builder.md>
- [27] GH Archive, *Gh archive: Github archive data*, 2026. [Online]. Available: <https://www.gharchive.org/>
- [28] deps.dev, *Deps.dev api*, Accessed: 2026-02. [Online]. Available: <https://docs.deps.dev/api/v3/>
- [29] OSS Review Toolkit, *Analyzer*, Accessed: 2026-02. [Online]. Available: <https://oss-review-toolkit.github.io/ort/docs/tools/analyzer>
- [30] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 35, 2022.
- [31] Stack Overflow. “Technology | 2025 stack overflow developer survey.” Accessed 2026-06-04. [Online]. Available: <https://survey.stackoverflow.co/2025/technology>
- [32] GitHub. “Octoverse: The state of open source.” Accessed 2026-06-04. [Online]. Available: <https://octoverse.github.com/>

A Appendix

A.1 Observable Health Indicator Categories

Table A.1 summarizes the categories of observable health indicators considered in this thesis (Section 2.2): an example of the signals in each category, the public source they are derived from, and their expected relevance to inactivity risk. The categories motivate the feature groups implemented in Section 4.7; the exact per-feature definitions are given in Section A.2 below. Security-practice indicators are listed for completeness but, as discussed in Section 3.3, are kept as parallel triage context rather than as inputs to the inactivity model.

Table A.1: Categories of observable health indicators and their expected relevance

Category	Example signals	Public source	Expected relevance
Commit activity	Commits over recent windows; active commit months; days since last commit	GH Archive push events	Direct evidence of ongoing development
Contributor activity	Recent and new contributors; contributor concentration	GH Archive commit actors	Team breadth and single-maintainer (bus-factor) risk
Issue and PR activity	Opened/closed issues; PR merge ratio; response and merge latency	GH Archive issue/PR events	Maintenance interaction and responsiveness
Release activity	Releases; days since last release; versions published	GH Archive releases and package registry	Whether usable versions still reach users
Age and popularity	Repository and package age; stars; forks; popularity tier	GitHub metadata and package registry	Maturity and reach context
Security practice	Selected OpenSSF Scorecard checks	OpenSSF Scorecard	Parallel security-posture context, not an inactivity predictor

A.2 Full-History Feature Reference

This section expands the feature engineering step of Section 4.7.

The full-history model consumes a feature vector of 60 variables under feature set `feature-set-v4-full-history`. These come from two sources, documented separately below: 43 *reconstructed historical* features, derived from the GH Archive event history before t and summarized by group in Table A.2 and individually in Table A.3; and 17 *current-snapshot* features, obtainable from a single observation of the repository and package metadata, listed in Table A.4. The cold-start regime uses exactly the 17 current-snapshot features and none of the historical ones, which is what makes it applicable to repositories with no reconstructed history. Table A.3 additionally lists two variables that are computed and exported for inspection but are *not* part of the 60-variable model vector: `repo_archived_at_obs`, excluded so that archival cannot become a shortcut label, and `repo_age_days`, which is identically zero under the reproducible offline build because that build carries no repository creation date (Section 4.4.1). Because the feature names are compact identifiers whose exact meaning is not obvious to a reader who is new to the artifact, this appendix documents each feature individually. The identifiers used here are the same strings that appear in the training code, in the exported model artifacts, and in the runtime interface, so the tables double as a shared glossary across the thesis and the software.

Four conventions apply throughout and explain the recurring suffixes in the names. First, a numeric suffix denotes a trailing time window that ends at the observation time t : `_30d`, `_90d`, and `_365d` count events in the 30, 90, and 365 days immediately before t , and `_at_obs` denotes a state measured exactly at t . Second, all commit- and contributor-based counts use a human-actor filter, so activity generated by bots does not inflate the maintenance evidence. Third, wherever an absolute count would not be comparable across projects of very different sizes, the feature set also provides a relative or trend-based form (a ratio, a share, or a year-over-year change), because a large project and a small project can be equally well or badly maintained at very different absolute volumes. Fourth, a feature that is *undefined* for a repository is distinguished from one that is genuinely zero (Section 4.7.1): the “days since” features are right-censored at the age of the observable record, and the latency medians are left undefined and imputed by the model artifact rather than being set to zero, which would read as an instantaneous response. Every value is computed from information available at or before t only; the reasoning behind this restriction, and the distinction between it and the legitimate activity-persistence effect, is discussed in Section 4.9.

Table A.2: Implemented full-history feature groups

Feature group	Example features
Dependency context	Dependency count of the resolved package version at the observation time.
Commit activity	Commits in the last 30, 90, and 365 days; active commit months; days since last commit.
Contributor activity	Contributors in the last 90 and 365 days; new contributors; contributor concentration.
Issue activity	Opened and closed issues; issue closure ratio; issue backlog growth; stale open issues.
Pull request activity	Opened, merged, and closed-unmerged pull requests; pull request merge ratio; stale open pull requests; pull-request response and merge latency.
Release activity	Releases in the last 365 days; days since last release; versions published in the last 365 days; recent release flag.
Age and popularity proxies	Package age; repository age; stars and forks observed in the available historical data; popularity tier.
Derived risk proxies	Activity drop; contributor drop; release gap risk; concentration risk score.
Responsiveness and backlog pressure	Issue first-response time; issue resolution time; stale issue share; pull-request response and merge latency.

Table A.3: Reconstructed historical feature reference (feature-set-v4-full-history): exact definition and rationale for each of the 43 historical model inputs, plus the two diagnostic-only variables `repo_archived_at_obs` and `repo_age_days`.

Feature	Definition (all measured at or before t)	Why it is included
<i>Commit activity</i>		
<code>commits_30d</code>	Number of human commits in the 30 days before t .	Short-term development pulse; captures whether the project is moving right now.
<code>commits_90d</code>	Number of human commits in the 90 days before t .	Quarter-scale activity level that is less noisy than the 30-day count.
<code>commits_365d</code>	Number of human commits in the 365 days before t .	Annual development volume; also the baseline against which the year-over-year drop is measured.

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
active_commit_months_365d	Count of distinct calendar months (0–12) in the last year that contain at least one human commit.	Separates steady maintenance from a single burst: 300 commits in one month is a weaker health signal than the same work spread across ten months.
days_since_last_commit	Days between t and the most recent human commit; right-censored at the age of the observable record when no commit is ever observed.	Direct staleness signal; a long silence is one of the strongest indicators of impending inactivity.
<i>Contributor activity and concentration</i>		
contributors_90d	Distinct human contributors in the last 90 days.	Breadth of recent participation; a proxy for the size of the active team.
contributors_365d	Distinct human contributors in the last 365 days.	Annual contributor base; the reference for the contributor drop.
new_contributors_365d	Contributors active in the last year who never committed before the window started.	Measures inflow of fresh maintainers; a project with no newcomers is more exposed to attrition.
top1_contributor_commit_share_365d	Share (0–1) of the last year’s commits made by the single most active contributor.	Key-person dependency: a high share means the project is fragile if that one person leaves.
top2_contributor_commit_share_365d	Combined commit share of the two most active contributors in the last year.	Detects a very small core team even when no single person dominates.
contributor_concentration_index	Herfindahl index: sum of squared per-contributor commit shares over the last year.	A single scalar for how concentrated contributions are (1.0 = one person, near 0 = evenly spread).
maintainer_concentration_flag	1 if the top contributor share ≥ 0.7 or the top-two share ≥ 0.85 , else 0.	Binary bus-factor alarm for the clearly single-maintainer case.
<i>Issue activity and backlog</i>		
opened_issues_90d	Issues created in the last 90 days.	Inbound user demand and engagement with the project.
closed_issues_90d	Issues closed in the last 90 days.	Maintainer throughput on the issue tracker.

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
<code>issue_closure_ratio_90d</code>	Closed divided by opened issues over the last 90 days.	Whether maintainers keep pace with inflow, expressed relatively so it is comparable across project sizes.
<code>issue_backlog_growth_90d</code>	Relative change in the open-issue count between $t - 90$ days and t .	A backlog that is trending upward signals that maintainers are losing control; size-normalized.
<code>stale_open_issues_count_at_obs</code>	Issues opened before $t - 90$ days that are still open at t .	Long-unaddressed items indicate neglect of the tracker.
<i>Pull-request activity</i>		
<code>opened_prs_90d</code>	Pull requests created in the last 90 days.	Contribution inflow from outside the core team.
<code>merged_prs_90d</code>	Pull requests merged in the last 90 days.	Direct evidence that maintainers are still integrating outside work.
<code>closed_unmerged_prs_90d</code>	Pull requests closed without merging in the last 90 days.	A high count relative to merges can signal rejection or disengagement from contributions.
<code>pr_merge_ratio_90d</code>	Merged divided by opened pull requests over the last 90 days.	Responsiveness to contributors, expressed relatively.
<code>stale_open_prs_count_at_obs</code>	Pull requests opened before $t - 90$ days that are still open at t .	Ignored contributions discourage further contribution and indicate maintainer absence.
<i>Release activity</i>		
<code>releases_365d</code>	GitHub releases published in the last 365 days.	Shipping cadence: whether usable versions still reach users.
<code>days_since_last_release</code>	Days since the most recent release or package version; right-censored at the age of the observable record when no release is ever observed.	Release staleness; complements commit staleness for projects that ship rarely.
<code>versions_published_365d</code>	Package-registry versions published in the last 365 days.	Package-level shipping, which can differ from repository releases.
<i>Age and popularity proxies</i>		
<code>package_age_days</code>	Days from the first published package version to t .	Maturity context: young and long-established packages have different baseline inactivity risk.

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
repo_age_days	Days from repository creation to t .	Diagnostic metadata only, and not a model input: the offline build supplies no creation date, so this column is identically zero (Section 4.4.1).
stars_total_at_obs	Cumulative stars observed up to t (a lower-bound proxy reconstructed from archive events).	Popularity and visibility; more attention tends to correlate with continued maintenance.
forks_total_at_obs	Cumulative forks observed up to t .	Reuse and derivative activity around the project.
dependency_count_at_obs	Declared dependency count of the resolved package version at t .	Proxy for maintenance burden and integration complexity.
popularity_tier_at_obs	Low / medium / high tier (0/1/2) derived from star and fork thresholds.	Coarse popularity bucket, because inactivity dynamics differ between obscure and widely used projects.
<i>Diagnostic and binary flags</i>		
repo_archived_at_obs	1 if the repository is archived by t , else 0.	Diagnostic metadata only, and not a model input; already-archived rows are excluded from fitting so archival cannot become a shortcut label.
has_recent_release_flag	1 if any release or package version appeared in the last 365 days.	Simple “is it still shipping” indicator that is robust when exact counts are sparse.
has_recent_pr_merge_flag	1 if any pull request was merged in the last 90 days.	Simple “are maintainers still integrating” indicator.
has_issue_activity_365d	1 if at least one issue was opened in the last 365 days, else 0.	Makes “no issue tracker activity at all” an explicit signal, so its absence cannot reach the model disguised as a zero-valued response latency.
has_pr_activity_365d	1 if at least one pull request was opened in the last 365 days, else 0.	The pull-request counterpart; distinguishes a project nobody contributes to from one that merges contributions instantly.
<i>Derived risk proxies</i>		

continued on next page

Table A.3 continued from previous page

Feature	Definition (all measured at or before t)	Why it is included
<code>activity_drop_365d_vs_prev_365d</code>	Relative decline in commits from the previous year to the last year (positive = fewer commits).	Captures deceleration, enabling earlier detection of decline than the absolute level alone.
<code>contributors_drop_365d_vs_prev_365d</code>	Relative decline in the contributor count from the previous year to the last year.	Detects a shrinking maintainer base before commits fully stop.
<code>release_gap_risk</code>	Score in $[0,1]$ for how overdue the next release is relative to the project's own release cadence (age-based when no release history exists).	Normalizes release silence against the project's normal rhythm rather than an absolute threshold.
<code>concentration_risk_score</code>	Composite score in $[0,1]$ combining top-1 and top-2 shares, the Herfindahl index, and the concentration flag.	A single interpretable bus-factor risk value that consolidates the concentration features.
<i>Responsiveness and backlog pressure</i>		
<code>issue_first_response_median_days_365d</code>	Median days from issue creation to the first maintainer response over the last year; <i>undefined</i> if no issue received a response in the window.	Slowing first responses indicate disengaging maintainers, independently of resolution speed.
<code>issue_resolution_median_days_365d</code>	Median days from issue creation to closure over the last year; <i>undefined</i> if no issue was closed in the window.	How quickly reported problems are actually resolved.
<code>stale_issue_share_at_obs</code>	Stale open issues divided by the current open backlog.	The fraction of the backlog that is old, expressed relatively so it is comparable across sizes.
<code>pr_response_median_days_365d</code>	Median days to the first response, merge, or close on pull requests over the last year; <i>undefined</i> if no pull request was responded to.	Responsiveness to contributors, the pull-request counterpart to issue first response.
<code>pr_merge_latency_median_days_365d</code>	Median days from creation to merge for merged pull requests over the last year; <i>undefined</i> if no pull request was merged.	Integration speed for work that is ultimately accepted.

The remaining 17 model inputs are *current-snapshot* features (Table A.4). They require only a single observation of the repository and its package metadata, with no event history, which is precisely why they constitute the entire cold-start feature set (`feature-set-v4-cold-start`). They are also part of the full-history vector, so a full-history model sees both tables.

Table A.4: Current-snapshot feature reference: the 17 features obtainable without event history. These form the complete cold-start vector and are also included in the full-history vector.

Feature	Definition (measured at t)	Why it is included
has_repository_mapping	1 if the package resolves to a source repository.	Without a repository, all activity signals are unavailable; the model must know this.
is_direct_dependency	1 if the package is a direct rather than transitive dependency.	Triage context: direct dependencies are more readily replaced.
last_push_age_days	Days since the most recent push to the default branch; right-censored when no push is observed.	The cold-start counterpart to commit staleness.
last_release_age_days	Days since the most recent release; right-censored when none is observed.	Shipping staleness without needing release history.
release_cadence_days	Mean days between consecutive releases; <i>undefined</i> with fewer than two releases.	The project’s normal shipping rhythm, against which silence is judged.
recent_contributors_90d	Distinct contributors in the last 90 days.	Breadth of the currently active team.
contributor_concentration	Commit share of the most active contributor.	Key-person dependency without full history.
open_issue_growth_90d	Relative change in the open-issue count over 90 days.	Whether the backlog is trending upward.
pr_response_median_days	Median days to first response on pull requests; <i>undefined</i> if none were responded to.	Responsiveness to contributors.
stars_log1p	$\log(1 + \text{stars})$ at t .	Popularity on a compressed scale; raw counts span several orders of magnitude.
forks_log1p	$\log(1 + \text{forks})$ at t .	Reuse and derivative activity, likewise compressed.
open_issues_log1p	$\log(1 + \text{open issues})$ at t .	Backlog size, compressed and comparable across project sizes.
ecosystem_npm, ecosystem_pypi, ecosystem_go, ecosystem_maven, ecosystem_other	Five mutually exclusive one-hot indicators of the package ecosystem.	Baseline inactivity rates and release conventions differ by ecosystem.

A.3 Foundation Seed Composition

The foundation seed is generated from the GitHub search API over public, non-fork repositories with a license object and at least 100 stars, drawn across active, dormant, and archived strata. The strata are sampling provenance only and never enter the supervised label (Section 4.4). Table A.5 contrasts the target frame with the seed as actually built. Two strata could not be filled from the search frame, and a fallback stratum supplied the shortfall with active repositories, which is why the realized seed is majority-active despite a frame designed to oversample inactivity.

Table A.5: Foundation seed strata: target frame versus realized seed (5,000 repositories).

Stratum	Target	Realized	Target share	Realized share
Active	2,250	2,601	45.0%	52.0%
Dormant	1,750	1,399	35.0%	28.0%
Archived	1,000	1,000	20.0%	20.0%
Total	5,000	5,000	100%	100%

The realized active count includes a 506-repository fallback stratum that was used to reach the target size after the `active-foundation` stratum returned 845 of 1,000 requested repositories and the `dormant-foundation` stratum returned 649 of 1,000.

A.4 Undefined Feature Handling

This section gives the exact rules summarized in Section 4.7.1.

Censoring bound. The age of the observable record is the interval from the later of the repository creation date and the start of archive coverage up to the observation time t . When a repository has no observed commit (or no observed release), the corresponding “days since” feature is set to this bound, which is the largest value consistent with the data. When the record begins at or after t — that is, when nothing at all was observable before the observation date — there is no interval over which the event could have been missed and therefore no bound to censor against; the feature is then undefined rather than zero. In the reported dataset this applies to 7,687 of 50,000 rows, whose repositories have no GH Archive event before their observation date.

Undefined values. The four responsiveness medians (`issue_first_response_median_days_365d`, `issue_resolution_median_days_365d`, `pr_response_median_`

`days_365d`, `pr_merge_latency_median_days_365d`) and the two censored age features are exported as absent keys rather than as `null` or 0, so that no consumer can silently read them as zero. Downstream, the gradient-boosted trees route undefined values natively through XGBoost’s missing-value branch, while the logistic-regression and neural-network models substitute the per-feature training-set median, which is stored in the model artifact alongside the standardization parameters and re-applied identically at inference. Because a substituted median standardizes to approximately zero, an undefined feature contributes little to the linear score rather than pulling it toward the favourable extreme.

Why it matters. Before this correction, an undefined latency was stored as 0.0. Since 0 days is the best possible response time, roughly half of the dataset encoded “this project has no issues or pull requests at all” as “this project responds instantaneously”. The variable was consequently a strong predictor of inactivity with an inverted meaning, and the feature attributions of Section 6.3.1 reflected data availability rather than maintenance behaviour. The same defect made `days_since_last_commit` non-monotonic, which in turn depressed the trivial recency baseline against which the learned models are compared (Section 6.1.1).

Offline-build consequences. Because the reported dataset is built through the reproducible offline path (Section 4.4.1), the repository creation date, archival timestamp, and fork flag are never populated, and three consequences follow. First, `repo_age_days` is identically zero and is therefore excluded from the model vector and from any age analysis. Second, `repo_archived_at_obs` is identically zero, so the archival condition of the maintenance label never fires (Section 4.3) and the rule excluding already-archived rows from fitting removes none. Third, build-time fork exclusion is inert, though the seed is generated with a non-fork filter so forks are already absent. Populating these attributes would require live enrichment of every seed repository and is noted as future work in Chapter 9.

A.5 Implementation Detail

This appendix collects the engineering detail of the artifact that is needed to reproduce or operate it, but that is not required to follow the scientific argument in Chapter 5. The cooperating system components referenced in Section 5.1 are summarized in Table A.6.

Table A.6: Implemented system components

Component	Technology	Responsibility
Scoring and training workspace	Python	Historical dataset builder, feature engineering, model training, model artifacts, and the runtime scoring service.
Backend API	Go	Analysis orchestration, provider enrichment, durable job processing, and model-artifact scoring calls.
Analyst frontend	TypeScript / Next.js	Submission, repository overview, risk views, and machine-learning evaluation dashboards.
Shared schemas	TypeScript	Reusable data contracts shared by the frontend.
Orchestration scripts	Node.js	Seed generation, dataset building, notebook execution, and artifact staging commands.
Deployment assets	Docker / Kubernetes	Container images, Compose, and the Argo CD-managed Kubernetes deployment.

A.5.1 Technology Selection

This subsection expands the component overview of Section 5.1.

The components use different programming languages, each selected for the part of the system where it is most appropriate.

Python is used for all data engineering and machine learning, including the historical dataset builder, the feature and label computation, the model implementations, and the scoring service. Python was chosen because the libraries this thesis depends on — `numpy` for the from-scratch model implementations and `xgboost` for the gradient-boosted trees — are native to its ecosystem, and because it remains among the most widely used languages for data work [31], which eases reproduction of the analytical core. The scoring service is implemented with FastAPI.

Go is used for the backend orchestration service. The backend is primarily a concurrent input/output workload: it calls several external providers and processes durable jobs. Go’s static typing, its straightforward concurrency model, and its suitability for long-running network services make it a good fit for this role.

TypeScript with the Next.js framework is used for the analyst frontend, and TypeScript is also used for the shared data contracts. TypeScript provides type safety for the user interface and shares type definitions with the backend contracts. The frontend addition-

ally relies on the broader JavaScript and Node.js tooling ecosystem, which is among the most widely used on GitHub [32]. This is a tooling choice rather than a statement about the ecosystems the artifact scores: the dataset seed and the demonstrator in Chapter 7 are Python-centric, and the model itself is ecosystem-agnostic.

Node.js additionally hosts the orchestration command-line scripts under `scripts/ml`. These scripts coordinate the higher-level workflow steps, such as generating the repository seed, building the dataset, executing the training notebook, and staging artifacts. They wrap the underlying Python pipeline so that the full reproducible workflow can be triggered through consistent `npm` commands.

A.5.2 Dataset-Builder Modules

This subsection expands the dataset builder of Section 5.2.

The historical dataset builder is divided into focused modules, summarized in Table A.7, so that ingestion, event parsing, feature computation, and label computation can be tested independently.

Table A.7: Dataset builder modules

Module	Responsibility
<code>adapters</code>	deps.dev, GitHub, and npm/PyPI registry adapters used for package-to-repository resolution and stable metadata.
<code>events</code>	GH Archive parsing into normalized repository histories of commits, issues, pull requests, releases, stars, and forks.
<code>features</code>	Observation-time feature computation from history before t .
<code>labels</code>	12-month maintenance and inactivity label computation from the future window.
<code>sampling</code>	Candidate sampling and popularity-tier derivation.
<code>pipeline</code>	The <code>DatasetBuilder</code> orchestrating ingestion, snapshots, labels, and export.
<code>storage</code>	JSON and JSON Lines reading and writing of the intermediate artifacts.
<code>cli</code>	The command-line entry point used by the orchestration scripts.

The `DatasetBuilder` class executes the construction workflow described in the methodology. The `ingest_candidates` step loads and de-duplicates seed candidates, resolves each package to a GitHub repository, and writes the `repositories.jsonl`, `packages.jsonl`, and `package_repository_links.jsonl` files. Candidates that cannot be mapped to a GitHub repository, and forks when forks are excluded, are dropped at this stage. The `build_snapshots` step iterates over quarterly observation dates and,

for each repository, constructs an `ObservationSnapshot` with explicit feature and label windows before computing the observation-time feature row. The `build_labels` step computes the future-window label for every snapshot, and `export_training_dataset` assembles the final training rows and the runtime repository feature cache.

A.5.3 Dataset-Quality Reporting

To make the per-run documentation items required by the methodology (Section 4.6) reproducible rather than transcribed by hand, a small `dataset_quality` module computes a structured quality report from the exported snapshot rows. It produces three views. The first is row accounting: the total, labeled, and unlabeled counts, the inactive and active label balance, the number of already-archived-at-observation rows that are excluded from fitting, and the completeness signals introduced above, including the `repo_silent_before_horizon` count that quantifies how many labeled examples the corrected coverage gate retains. The second is label balance per slice, broken down by ecosystem and by popularity tier, which exposes where labeled examples are concentrated. The third is per-feature missingness, derived from the missing-signal annotations already attached during feature extraction, so that a feature is reported as missing only when its underlying observation-time signal was unavailable rather than genuinely zero. The report is rendered in the training notebook alongside the existing coverage summary, and the row accounting deliberately records whether label-completeness signals are present in the export so that a value of zero is not misread as an absence of incomplete windows when an older export simply did not carry them.

A.5.4 Model Configurations

This subsection expands the model families introduced in Section 5.3 and specified in Section 4.10.

The Logistic Regression model is implemented directly in Python. It standardizes each input feature using training-set means and standard deviations, stores these standardization parameters with the model, and fits the coefficients by gradient descent on the class-weighted log-likelihood. Class-balanced sample weights counter the label imbalance, and an L2 penalty regularizes the coefficients. Because the model stores explicit per-feature coefficients, it doubles as an interpretable baseline whose weights can be inspected to check whether the engineered features behave as expected. This interpretability is what made it useful during the feature-set refinement described in Section A.5.12.

The neural network is a multilayer perceptron implemented on top of numpy, without a deep-learning framework. It uses two hidden layers of 64 and 32 units with ReLU activations and a sigmoid output, standardizes its inputs in the same way as the Logistic Regression model, and initializes its weights with Xavier-uniform values under a fixed random seed for reproducibility. Training uses mini-batch stochastic gradient descent on the binary cross-entropy objective, with balanced class weights and L2 regularization. The default configuration matches the methodology: a learning rate of 0.01, 150 epochs, a batch size of 256, and an L2 penalty of 0.001. The forward and backward passes are written explicitly, which keeps the model transparent and removes any heavy training dependency from the artifact.

The XGBoost model wraps the `xgboost` library. It trains a gradient-boosted tree ensemble with a binary-logistic objective and the same class-balanced sample weights as the other models. The default configuration uses 80 boosting rounds, a maximum tree depth of three, a learning rate of 0.08, row and column subsampling of 0.9, L2 leaf regularization, the histogram tree method, and a fixed seed. After training, the trained booster is serialized to its JSON representation and stored inside the model artifact, and the gain-based feature importances are computed and normalized so they can be displayed in the evaluation dashboard.

A.5.5 Artifact Serialization

This subsection expands the artifact format referred to in Section 5.3.

Each trained model is serialized to a self-describing JSON artifact. The artifact stores the model family and version, the ordered feature names, the model parameters (coefficients and standardization for Logistic Regression, weights and biases for the neural network, or the serialized booster for XGBoost), the calibration bins, the decision threshold, the feature-set version, and the training timestamp. Because the feature names and standardization are part of the artifact, the runtime service can construct the feature vector in the correct order and apply the same transformations without sharing code paths with the training process. Each training run is written to a run file under the training runs directory together with the dataset hash and the held-out metrics, and a `latest-run.json` pointer records the most recently produced run.

A.5.6 Training Orchestration

This subsection expands the training workflow of Section 5.4.

The headless command `npm run ml:train` executes the training notebook non-interactively and writes the run artifacts and the latest-run pointer, so that the same notebook is used for both interactive exploration and reproducible artifact generation. The end-to-end command `npm run ml:bootstrap` chains dataset construction and notebook-based training, and the foundation variants of these commands use the preserved 5,000-repository seed and the filtered GH Archive cache for the thesis-grade run. The orchestration scripts wrap the Python pipeline rather than reimplementing it; they pass through the seed file, the GH Archive source, the output directories, the observation window, and the dataset-size guardrails, which keeps the reproducibility parameters explicit at the command line.

Listing A.1: Headless notebook-based training

```
1 npm run ml:train
```

A.5.7 Artifact Staging and Promotion Gate

This subsection expands the promotion gate summarized in Section 5.4.

Trained artifacts are not deployed automatically. The staging command `npm run ml:stage-training` validates a candidate training bundle and promotes it into the deployed bundle under `deployment/training` only if it passes a set of quality gates. This gate is the safety boundary between experimentation and deployment.

The staging script first validates the dataset itself: the snapshots must be non-empty, every labeled snapshot must come from a real GitHub repository, and the bundle must contain at least a configured minimum number of distinct repositories and distinct inactive repositories. It then validates the model run artifacts: all required model artifacts must be present and complete for the latest dataset hash, and their feature versions must match the expected full-history and cold-start feature sets. This prevents a partial or inconsistent bundle, for example one missing the cold-start models, from reaching deployment.

The most important gate is the regression comparison. Before promotion, the candidate artifacts are compared against the currently deployed baseline. For each required model, the script checks that ROC-AUC has not dropped, and that the Brier score has not increased, by more than a configured tolerance. If any required model fails this comparison, promotion is rejected and the existing staged bundle is retained. When promotion succeeds, the script normalizes the snapshot file, the run artifacts, and the repository feature cache to their runtime paths and writes them into the deployed

bundle.

A.5.8 Scoring Service

This subsection expands the scoring component of Section 5.1, and implements the undefined-feature rules of Section 4.7.1.

The scoring service is a FastAPI application under `mltraining/scoring/app`. It exposes a small internal interface: health and readiness probes, a `/features/extract` endpoint for feature extraction, and the `/score/model` endpoint that scores a batch of dependencies with a supplied model artifact. The service does not own a model registry; instead, the backend passes the staged artifact in the scoring request, which keeps the scoring service stateless and makes the deployed model an explicit input rather than hidden state.

For each dependency, the service builds the feature vector in the artifact's feature order, imputing undefined signals from the training-set medians stored with the artifact (Section 4.7.1) and recording which signals were missing. It deserializes the artifact according to its model family, computes the raw probability, and applies the stored calibrator to obtain the calibrated inactivity probability. This probability is scaled to a 0–100 inactivity-risk score, and a complementary 12-month maintenance-outlook score is derived as its inverse. The service assigns a discrete risk bucket and an action level from the score, where the action level is one of `monitor`, `review`, or `replace_candidate`, so that the continuous score maps onto the triage actions used in the demonstrator.

The service keeps the inactivity prediction and the security posture strictly separate, exactly as required by the methodology. The OpenSSF Scorecard signal is summarized into a parallel security-posture score and is never fed into the predictive feature vector [18]. A confidence value is computed from the completeness of the available input signals and the decisiveness of the prediction, and explicit caveats are attached when calibration bins are missing, when the artifact's feature version is unexpected, or when input signals had to be imputed. Finally, the service returns human-readable explanation factors and evidence items so that an analyst can see which signals shaped the score rather than only the number itself.

A.5.9 Backend Orchestration Service

This subsection expands the backend component of Section 5.1 and realizes the two prediction regimes of Section 4.8.

The backend is a Go service under `backend/api`. It exposes the public REST API for analyses, repositories, jobs, and the training dataset and run endpoints. Its central responsibility is to turn a submission into an enriched, scored analysis reliably, even in the presence of slow or failing external providers.

Analyses are processed through a durable job queue backed by PostgreSQL. When a submission arrives, the service creates an analysis record and a job record in a single transaction and returns immediately. A background worker leases the next pending job, processes it, and persists the result. If processing fails, the job is retried with a bounded number of attempts and a retry delay, and it is only marked as failed once the attempts are exhausted. This design makes the analysis lifecycle resilient and observable, and it decouples the user-facing request from the slower enrichment and scoring work. For repository submissions, the service also reuses a recent completed analysis when one exists for the same repository and the same staged model set, which avoids redundant re-enrichment.

During processing, the worker enriches each repository and scores it. Enrichment fetches stable repository metadata from GitHub and retrieves the OpenSSF Scorecard signal, attaching the raw signals to the repository for display. Each enrichment provider is implemented as a separate adapter, so a single failing provider degrades the available signals rather than the whole analysis.

The backend implements the two prediction regimes from the methodology at runtime. When it enriches a repository, it attempts to look the repository up in the staged repository feature cache. On a cache hit, the repository is scored in the full-history regime using the cached observation-time features; on a miss, it falls back to the cold-start regime using only approximate features derived from live metadata, and a caveat records this. When several model artifacts are staged for the same dataset hash and regime, the backend scores the repository with each and combines the results into an ensemble profile by averaging the scores, while also retaining the per-model outputs and adding a model-agreement explanation factor that reports the spread between models.

A.5.10 Analyst Frontend

This subsection expands the frontend component of Section 5.1; the analyst workflow it serves is demonstrated in Chapter 7.

The frontend is a Next.js application under `frontend/web` that uses the React component model and the application router. It provides the analyst-facing workflow and the evaluation dashboards used during the thesis. Submission is handled through a

form that accepts one or more repository URLs or a demo analysis. Once an analysis completes, the dashboard presents the repository overview with risk distribution and filtering, and a dedicated view renders the per-repository detail card with its risk profile, caveats, and evidence.

A second area of the frontend is the machine-learning evaluation surface. These pages read the training dataset and run endpoints exposed by the backend and visualize them with purpose-built charts, including a calibration curve, a model-metric comparison across the trained models, a Logistic Regression coefficient view, and a training-metric history. These views were used directly to inspect and report model behavior during the thesis.

A.5.11 Deployment

This subsection expands the deployment path of Section 5.4.

The artifact is containerized for both local and production-style operation. Canonical image definitions live under `deployment/docker`, and a Docker Compose file orchestrates PostgreSQL, the Go backend, the scoring service, and the frontend for local development; a single `npm run dev` command builds and starts the full stack and waits for the services to become healthy. For production-style operation, the project is deployed to a k3s cluster managed by Argo CD, with container images published to a registry and the application served behind a public domain [25]. Because the deployed model bundle is a versioned set of files under `deployment/training`, deployment and model promotion remain auditable: the running service uses exactly the artifacts that passed the staging gate described in Section A.5.7.

A.5.12 Feature-Set Refinement

This subsection gives the details of the first design-cycle refinement summarized in Section 5.2.

The feature set used for training was refined iteratively, as described in the methodology’s formative development step (Section 4.2). The first feature set contained variables that turned out to be poor point-in-time predictors. A representative example is a raw open-issue count: large, actively maintained projects can carry very large issue backlogs, so a model can wrongly associate a high absolute issue count with risk, even though the project is healthy. The revised feature set therefore replaces raw counts with relational and trend-based variables, such as issue closure ratio, issue backlog growth, stale-issue

share, and pull-request response and merge latency, which are more comparable across projects of different sizes.

A second class of problems came from repository identity. Some packages point to mirror repositories rather than to the upstream development repository, which can make a project look inactive even though active development happens elsewhere. The mapping logic therefore prefers explicit seed and deps.dev mappings and resolves the primary package-to-repository link per repository, rather than treating every registry entry as an independent signal. Finally, the revised feature set keeps `repo_archived_at_obs` only as diagnostic metadata and excludes already-archived observation rows during model fitting, so that the visible archival flag cannot become a shortcut label.

A.5.13 Slice Evaluation

This subsection gives the mechanics of the slice-evaluation instrument introduced in Section 5.3; the resulting figures are reported in Section 6.3.3 and tabulated in Section A.10.

A single aggregate metric can hide a subgroup on which the model performs poorly, because a strong majority subgroup can mask a weak minority one. A slice-evaluation helper therefore re-scores the same held-out predictions within subgroups, reusing the existing metric implementation so that no separate evaluation logic is introduced. To make this possible the training pipeline also returns the per-row held-out predictions alongside the aggregate result; these are used only for analysis and are not written into the model artifact, so the serialized artifacts and the reported headline metrics are unchanged. The harness includes a consistency check that requires the aggregate over all slices to reproduce the model’s reported held-out ROC-AUC, so the per-slice figures are guaranteed to originate from the same execution as the headline metrics.

The subgroups are those that vary meaningfully in the foundation dataset: popularity tier, a cold-start-like versus has-history regime, and whether the repository was still active at the observation date. The last of these is the active-at- t subset referenced in the evaluation strategy (Section 4.11) and in the discussion of activity persistence in Chapter 8: restricting the evaluation to repositories that are still active at t separates genuine early-warning skill from the trivial tendency of already-quiet repositories to remain quiet. Two candidate dimensions are deliberately excluded because the sampling frame holds them constant. Ecosystem is excluded because the repository-first foundation seed assigns the same provenance ecosystem to every row; cross-ecosystem evaluation would require a package-first sampling frame and is left to future work.

Repository-age band is excluded because the offline build carries no repository creation date, so `repo_age_days` is identically zero and every row would fall into a single band (Section 4.4.1); reporting such a slice would suggest an age analysis that the data cannot support. Because slicing reduces the sample size, each slice is reported with its own row count, ranking metrics that are undefined for single-class slices are reported as absent rather than as a misleading value, and thin slices are interpreted with corresponding caution.

A.5.14 Repository-Disjoint Robustness Split

This subsection gives the mechanics of the repository-disjoint split introduced in Section 5.3; it addresses the repository-recurrence threat of Chapter 8 that arises from the split described in Section 4.6.

The reported evaluation uses a chronological split: rows are ordered by observation date and cut into training, validation, and test partitions. Because observations are taken quarterly, a repository contributes several snapshots over time, so the same repository can appear in both the training and the test partition, and the twelve-month label windows of its successive snapshots overlap. The held-out metric then partly reflects generalization to later observations of repositories already seen during training, which is optimistic relative to performance on genuinely new repositories. This is the deployment-realistic question — the system does re-score repositories over time — but it should not be the only question asked.

A further evaluation iteration therefore adds a repository-disjoint split that assigns every snapshot of a repository to a single partition, so that the held-out set contains only repositories absent from training. The split is selectable through a configuration option whose default preserves the chronological behavior, so the reported artifacts and headline metrics are unchanged, and the pipeline asserts that the partitions are repository-disjoint whenever the grouped split is used. Running the deployed full-history model under both splits lets the difference between them bound the optimism introduced by repository recurrence. In practice the held-out ROC-AUC decreases only marginally under the repository-disjoint split, which indicates that the model’s discrimination is not an artifact of memorizing individual repositories but generalizes to unseen ones; the exact figures for both splits are produced in the training notebook and reported in Chapter 6.

A.6 Dataset Construction Steps

This section expands the dataset construction workflow of Section 4.5; its software realization is described in Section 5.2.

The historical dataset builder creates its dataset through the sequence of intermediate steps summarized in Table A.8, writing the intermediate files listed below under the configured output directory so that the construction process is auditable and the generated dataset can be inspected before training.

Table A.8: Historical dataset construction workflow

Step	Description
1	Load repository candidates from the foundation seed.
2	Use the seed strata to obtain a mixture of active, dormant, and archived repository histories.
3	Resolve repository identities from explicit seed fields or package meta-data where required.
4	Enrich stable repository metadata through GitHub.
5	Parse GH Archive event files into normalized repository histories.
6	Build quarterly observation snapshots for each repository.
7	Compute observation-time features using only data available at or before t .
8	Compute 12-month labels using only the future window $(t, t + 12 \text{ months}]$.
9	Export the final snapshot dataset for the existing model-training workflow.
10	Write a repository feature cache for runtime full-history scoring.

The documented intermediate outputs are `repositories.jsonl`, `packages.jsonl`, `package_repository_links.jsonl`, `observation_snapshots.jsonl`, `snapshot_features.jsonl`, `snapshot_labels.jsonl`, `training_snapshots.json`, and `repository-feature-cache.json`.

A.7 Evaluation Metric Definitions

This section gives the formal definitions of the metrics named in the evaluation strategy (Section 4.11) and reported throughout Chapter 6.

All metrics are computed on the calibrated probabilities of the held-out test slice. Let N be the number of evaluation samples, $p_i \in [0, 1]$ the predicted probability of inactivity for sample i , and $y_i \in \{0, 1\}$ the true label (1 = inactive within twelve months). The

positive (inactive) base rate of the slice is

$$\bar{p} = \frac{1}{N} \sum_{i=1}^N y_i, \quad (\text{A.1})$$

reported as the *inactive 12m rate*.

Ranking. ROC-AUC is computed as the Mann–Whitney statistic over all positive–negative pairs, that is, the probability that a random inactive repository is scored above a random active one, with ties counted as one half:

$$\text{AUROC} = \frac{1}{N_+ N_-} \sum_{i: y_i=1} \sum_{j: y_j=0} \left(\mathbb{I}[p_i > p_j] + \frac{1}{2} \mathbb{I}[p_i = p_j] \right), \quad (\text{A.2})$$

where N_+ and N_- are the numbers of positive and negative samples. It is threshold- and calibration-independent: 0.5 is random ranking and 1.0 is a perfect ranking.

Probabilistic accuracy. The Brier score is the mean squared error between probability and outcome, and the log loss is the binary cross-entropy (with a small ε for numerical stability):

$$\text{Brier} = \frac{1}{N} \sum_{i=1}^N (p_i - y_i)^2, \quad (\text{A.3})$$

$$\text{LogLoss} = -\frac{1}{N} \sum_{i=1}^N \left[y_i \ln p_i + (1 - y_i) \ln(1 - p_i) \right]. \quad (\text{A.4})$$

Both are lower-is-better and penalize miscalibration; the log loss grows without bound as a confident prediction approaches the wrong outcome, so it punishes overconfidence more sharply than the bounded Brier score.

Calibration. The expected calibration error partitions $[0, 1]$ into $B = 10$ equal-width bins. For bin b with member set $\mathcal{B}_b$, let conf_b be the mean predicted probability and acc_b the empirical positive rate; the ECE is the sample-weighted average gap between predicted confidence and observed frequency:

$$\text{ECE} = \sum_{b=1}^B \frac{|\mathcal{B}_b|}{N} \left| \text{conf}_b - \text{acc}_b \right|. \quad (\text{A.5})$$

Thresholded classification. A decision threshold τ maps probabilities to decisions $\hat{d}_i = \mathbb{I}[p_i \geq \tau]$, yielding the counts of true and false positives and negatives (TP, FP, FN, TN).

From these,

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{F1} = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (\text{A.6})$$

The F1 score is the harmonic mean of precision and recall, so it cannot be made large by optimizing one at the expense of the other; τ is selected on the validation slice to maximize it.

Combined quality score. The quality score converts ranking and probabilistic accuracy into skill scores relative to the trivial base-rate predictor and averages them, weighting ranking at 0.6 and calibration at 0.4. With the climatological Brier score $\bar{p}(1 - \bar{p})$ and $\text{clip}_{[0,1]}$ clamping to the unit interval,

$$s_{\text{AUC}} = \text{clip}_{[0,1]} \left(\frac{\text{AUROC} - 0.5}{0.5} \right), \quad (\text{A.7})$$

$$s_{\text{Brier}} = \text{clip}_{[0,1]} \left(1 - \frac{\text{Brier}}{\bar{p}(1 - \bar{p})} \right), \quad (\text{A.8})$$

$$\text{Quality} = \text{clip}_{[0,1]} (0.6 s_{\text{AUC}} + 0.4 s_{\text{Brier}}). \quad (\text{A.9})$$

A value of 0 indicates no improvement over predicting the base rate and 1 indicates perfect ranking and calibration. Because both skill terms are measured against the slice base rate $\bar{p}$, the quality score is conditional on the sampled prevalence and is intended for model comparison rather than as a standalone performance guarantee.

A.8 Reproducibility

This section makes the dataset construction workflow of Section 4.5 reproducible, and records the identifying metadata of the run whose results are reported in Chapter 6.

The dataset builder and training pipeline are executed through repository scripts. The foundation seed can be generated with:

```
npm run ml:seed:foundation -- '
  --target-repositories 5000 '
  --github-token <TOKEN>
```

The larger foundation dataset can then be built and trained with:


```
npm run ml:bootstrap:foundation -- ‘  
  --github-token <TOKEN> ‘  
  --gharchive-source <GHARCHIVE_PATH>
```

The underlying full command path uses a generated foundation seed, a GH Archive source directory, a dedicated training output directory, a training snapshot export path, and a repository feature cache output path. The repository documentation recommends the foundation path for the thesis/training base, while the smaller default bootstrap path remains a local smoke-test workflow [26].

For each final training run, the following items should be documented:

- seed generation command and seed metadata,
- GH Archive coverage period,
- output directory,
- training snapshot path,
- repository feature cache path,
- dataset hash,
- model artifact path,
- model name and version,
- feature version,
- number of total, labeled, and unlabeled rows,
- train, validation, and test split sizes,
- held-out evaluation metrics.

The reported full-history run uses dataset hash:

```
2d29a928dac34ea1dab919f87e89386f2e6eac3bda59aee7a625c5ad0873b0d8
```

and exports the model artifact:

```
runs/20260709T205727480957Z-xgboost-full-history-2d29a928dac3.json
```

The corresponding source revision is archived at the commit referenced in the artifact citation [25], so that the reported run can be traced to a fixed state of the dataset builder and training pipeline.

A.9 Full Demonstrator Ranking

Table A.9 lists the complete ranked inactivity-risk view for all 30 dependency repositories of ArchipelagoMW/Archipelago produced by the deployed model in the demonstrator (Chapter 7); the body reports the flagged and medium-risk subset in Table 7.1.

The explanation factors of the highest-ranked repository, `bsdifff4` (`ilanschnell/bsdifff4`, risk 92.1; Section 7.2), taken directly from the scored output, illustrate the intended semantics: the two cold-start ensemble members predict 97.0% and 87.2% inactivity risk, the model-agreement factor reports the spread between them, and the input-completeness factor records that 75% of expected signals were available before imputation. The moderate confidence of 0.69 and the caveats attached to the score — that it came from the cold-start model, no full-history cache row being available, and that two release-related signals were imputed — are what present the flag as a prompt for manual review rather than a definitive judgement.

A.10 Supplementary Evaluation Tables

This section collects the detailed evaluation tables whose findings are reported and interpreted in Chapter 6. Table A.10 gives the full per-subgroup slice evaluation of the deployed `xgboost-full-history` model (Section 6.3.3), and Table A.11 gives the cross-test of the human-actor filter (Section 6.4).

A.10.1 Human-Actor Filter Ablation

This subsection expands the ablation summarized in Section 6.4. The full 5,000-repository dataset was rebuilt with the human-actor filter disabled through a single build flag, with every other setting identical, yielding 50,000 snapshots that align one-to-one with the filtered build. Three effects are examined.

Label corruption. Disabling the filter flips 357 of 50,000 labelled snapshots (0.71%) from inactive to active, and *none* in the opposite direction. The effect is strictly one-directional: a bot commit or merge inside the future window can satisfy the two-of-four maintenance rule, so an otherwise dormant repository is relabelled as maintained, whereas bots never make an active repository look inactive. Omitting the filter would therefore bias the target by systematically *masking* inactivity, which is the error direction that matters most for a risk signal.

Table A.9: Complete ranked inactivity-risk view for the dependency repositories of ArchipelagoMW/Archipelago, as produced by the deployed model. Risk is the 0–100 inactivity-risk score; “Conf.” is the per-prediction confidence; “Regime” is full-history (FH) or cold-start (CS).

Package	Source repository	Risk	Action	Conf.	Regime
bsdifff4	ilanschnell/bsdifff4	92.1	replace_candidate	0.69	CS
setproctitle	dvarrazzo/py-setproctitle	80.4	replace_candidate	0.61	CS
jinja2	pallets/jinja	77.1	review	0.77	CS
cymem	explosion/cymem	74.9	review	0.75	CS
markupsafe	pallets/markupsafe	74.2	review	0.75	CS
pymem	srounet/Pymem	54.3	review	0.47	CS
flask-compress	colour-science/flask-compress	48.5	monitor	0.45	CS
pony	ponyorm/pony	42.8	monitor	0.59	CS
flask-limiter	alisaiifee/flask-limiter	41.9	monitor	0.58	CS
kivymd	kivymd/KivyMD	41.8	monitor	0.47	CS
waitress	Pylons/waitress	41.3	monitor	0.59	CS
schema	keleshev/schema	37.3	monitor	0.78	CS
pyyaml	yaml/pyyaml	35.4	monitor	0.77	CS
orjson	ijl/orjson	31.1	monitor	0.61	CS
colorama	tartley/colorama	27.8	monitor	0.54	CS
pyshortcuts	newville/pyshortcuts	25.0	monitor	0.75	CS
werkzeug	pallets/werkzeug	24.3	monitor	0.76	CS
certifi	certifi/python-certifi	23.5	monitor	0.57	CS
platformdirs	tox-dev/platformdirs	22.6	monitor	0.77	CS
pathspec	cpburnz/python-pathspec	16.6	monitor	0.83	CS
kivy	kivy/kivy	15.0	monitor	0.85	CS
websockets	python-websockets/websockets	10.3	monitor	0.90	CS
flask	pallets/flask	9.5	monitor	0.91	FH
flask-cors	corydolphin/flask-cors	8.9	monitor	0.91	CS
typing-extensions	python/typing_extensions	5.8	monitor	0.95	CS
bokeh	bokeh/bokeh	5.3	monitor	0.71	CS
flask-caching	pallets-eco/flask-caching	5.0	monitor	0.95	CS
mistune	lepture/mistune	4.2	monitor	0.96	CS
psutil	giampaolo/psutil	3.4	monitor	0.72	CS
cython	cython/cython	2.6	monitor	0.98	CS

Table A.10: Held-out ROC-AUC and Brier score of `xgboost-full-history` within subgroups of the test slice. “Inactive rate” is the positive rate in the subgroup.

Dimension	Subgroup	n	Inactive rate	ROC-AUC
Popularity	high	2297	0.527	0.972
Popularity	medium	1753	0.778	0.964
Popularity	low	950	0.767	0.846
Activity at t	active@ t	1794	0.225	0.905
Activity at t	quiet@ t	3206	0.905	0.911
History regime	has-history	2295	0.350	0.921
History regime	cold-start-like	2705	0.924	0.929

Table A.11: Cross-test of the human-actor filter: training on bot-contaminated features versus clean features, both evaluated against the identical clean (human-filtered) label on the held-out slice. A lower ROC-AUC or higher Brier for the contaminated features means the filter helps.

Model	ROC-AUC		Δ ROC-AUC	Δ Brier
	clean	contam.		
logistic-regression-full-history	0.9494	0.9484	−0.0010	+0.0005
xgboost-full-history	0.9579	0.9569	−0.0010	+0.0001
logistic-regression-cold-start	0.9318	0.9308	−0.0011	+0.0005
xgboost-cold-start	0.9531	0.9524	−0.0006	+0.0010

Feature inflation. On the observation-time side, bots inflate the activity features substantially: with the filter disabled, `commits_365d` changes on 7,858 of 50,000 snapshots (15.7%) by a mean of +938 commits, though the median change is only +80, so the mean is driven by a small number of heavily automated repositories. `commits_90d` rises by a mean of +388 and `contributors_365d` by +1.4 distinct contributors, while pull-request *count* features are unaffected because they are not actor-filtered. A small number of bot-driven repositories would thus dominate the activity signal.

Predictive effect. A naive comparison, in which each model is scored against the labels of the dataset it was trained on, would show bots to be nearly harmless — but it is misleading, because the unfiltered model is then scored against a partially corrupted target. A cleaner test trains on the bot-contaminated *features* while evaluating against the clean, human-filtered *labels* (Table A.11): the contaminated features are consistently worse across all four families, though by a small margin (ROC-AUC falls by 0.0006–0.0011; Brier rises by up to 0.0010).

Declaration on the Use of Artificial Intelligence

Generative artificial intelligence was used during the development of the software artifact and during the preparation of this thesis. This declaration records that use in a form that can be independently verified.

The assistant used was *Claude Opus 4.8* (Anthropic), operated through the Claude Code command-line interface. It was used for software implementation, refactoring, test authoring, debugging, data analysis, and for drafting and revising prose in this document. Every contribution was reviewed, corrected where necessary, and accepted by the author, who takes full responsibility for the content of this thesis and for the correctness of the artifact.

Two categories of assistance are distinguished. Contributions that entered the version control history of the artifact are recorded as commit trailers of the form `Co-Authored-By: Claude Opus 4.8`, and are enumerated in Table A.12. The complete and current list can be regenerated from the repository at any time with:

```
git log --grep="Co-Authored-By: Claude" --format="%h %ad %s" --date=short
```

Contributions that did not produce a commit — interactive discussion, exploratory analysis, and drafting of text that was subsequently rewritten by the author — are not individually enumerable and are declared here collectively.

No generative model was used to produce, synthesize, or alter any empirical result. All figures reported in this thesis originate from the training and evaluation pipeline described in Chapter 5, are reproducible from the recorded dataset hash (Appendix A.8), and were not authored by a language model. No bibliographic reference in this thesis was generated by a language model; every entry was verified against its source.

At the time of writing, 16 of the 83 commits in the repository carry the co-authorship trailer.

Table A.12: Commits to the OSS Risk Radar repository co-authored by Claude Opus 4.8, in chronological order. Hashes are abbreviated to ten characters.

Commit	Date	Description
edcc7cfa87	2026-06-30	feat(training) : coverage-horizon label gate plus slice, grouped-split, and dataset-quality evaluation
bd5de37eb8	2026-06-30	docs : adjust thesis chapters based on model iteration change
188aabec79	2026-07-02	docs(web) : add architecture and API reference pages
7b1eb223c3	2026-07-02	refactor : scope analysis to a single repository
b7f1ef5071	2026-07-02	docs(thesis) : revise chapters for per-repo scope
38ff126d11	2026-07-02	docs(thesis) : add Provadis deckblatt and fill author metadata
cd557b9ea6	2026-07-02	docs(thesis) : drop redundant per-chapter summaries
9bda7ba437	2026-07-02	docs(thesis) : move implementation engineering detail to appendix
90708c2d94	2026-07-02	docs(thesis) : move metric formulas and runbook to appendix
67270a4b10	2026-07-02	docs(thesis) : trim redundant methodology table and anecdote
5fdb88fdb4	2026-07-08	docs(thesis) : shorten body via condensing and appendix relocation
cc4fa83116	2026-07-08	feat(web) : add repositories directory page and branding, slim radar
e2dfc31b5c	2026-07-08	fix(scoring) : regime-aware confidence and interpolated calibration
22429aae4b	2026-07-08	feat(ml) : add small OSS seed collection script and seed dataset
1b3b1e41c5	2026-07-08	chore : ignore agent worktrees and stray thesis PDF
6e850a2e71	2026-07-08	docs : add final-draft exposé PDF

Declaration of Authorship

I declare that I wrote this thesis independently, that I used no sources or aids other than those stated, and that all passages taken from other works, whether verbatim or in substance, are identified as such. The use of generative artificial intelligence is declared separately and in full in the Declaration on the Use of Artificial Intelligence (page 76). This thesis has not previously been submitted, in the same or a similar form, to any examination authority.

Place, date

Eric Krause